A Two-Stage Cascade and Safety-Constrained Language Model for Kidney-Risk Screening and Patient Education

by

Muhammad Sakib Sami Roll: 210525
Mohammad Walid Hasan Roll: 210536
Zunaid Ali Roll: 210548

A Research Project Report submitted to the
Institute of Information Technology
in partial fulfillment of the requirements for the degree of

Bachelor of Science in
Information and Communication Technology

Supervisor: M. Shamim Kaiser, PhD
Professor, Institute of Information Technology

Institute of Information Technology
Jahangirnagar University
Savar, Dhaka-1342

September 2026

DECLARATION

We hereby declare that this thesis is based on the results found by ourselves. Materials of work found by other researchers are mentioned by reference. This thesis, neither in whole nor in part, has been previously submitted for any degree.

Muhammad Sakib Sami	Mohammad Walid Hasan	Zunaid Ali
Roll: 210525	Roll: 210536	Roll: 210548

CERTIFICATE

This is to certify that the thesis entitled **"A Two-Stage Cascade and Safety-Constrained Language Model for Kidney-Risk Screening and Patient Education"** has been prepared and submitted by **Muhammad Sakib Sami**, **Mohammad Walid Hasan** and **Zunaid Ali** in partial fulfilment of the requirement for the degree of Bachelor of Science (honours) in Information and Communication Technology.

M. Shamim Kaiser, PhD
Supervisor

Accepted and approved in partial fulfilment of the requirement for the degree of Bachelor of Science (honours) in Information and Communication Technology.

Chairman	Member	Member

ACKNOWLEDGEMENTS

We feel pleased to have the opportunity of expressing our heartfelt thanks and gratitude to all who rendered their cooperation in making this report.

This research was performed under the supervision of **M. Shamim Kaiser, PhD**, Professor, Institute of Information Technology (IIT), Jahangirnagar University, Savar, Dhaka. Throughout the work he supplied us with books, journal articles and reference materials related to the present investigation, and gave generously of his time in reviewing our design decisions, our evaluation methodology and our statistical analysis. Without his guidance, kind support and patient supervision we could not have completed this project successfully within the available time. First and foremost, we wish to acknowledge our profound and sincere gratitude to him for his guidance, valuable suggestions, encouragement and cordial cooperation.

We express our utmost gratitude to the Director, IIT, Jahangirnagar University, for the advice and institutional support that encouraged us to complete the work within the time frame. We would also like to thank the other faculty members of IIT who have helped us directly or indirectly by providing their valuable support in completing this work.

We gratefully acknowledge the Centers for Disease Control and Prevention and the National Center for Health Statistics for making the NHANES 2021–2023 cycle publicly available, and the KDIGO, ADA, NKF/KDOQI and CDC organisations whose published clinical guidance forms the knowledge base of the education component of this system.

Finally, we would like to thank all the staff of IIT, Jahangirnagar University, our families and our friends, who have helped us with their encouragement and cooperation throughout the work.

ABSTRACT

Chronic kidney disease (CKD) affects an estimated 800 million people worldwide and remains undetected in most cases until an advanced stage, because the confirmatory markers—estimated glomerular filtration rate (eGFR) and urine albumin-to-creatinine ratio (UACR)—require a kidney-specific laboratory order that is rarely placed before a clinician already suspects disease. This work presents Nephro-AI, a cross-sectional kidney-risk screening and patient education system built exclusively on routine, low-cost measurements that a standard general checkup already produces. A two-stage cascade—a random-forest at-risk gate followed by a gradient-boosting high-risk discriminator—was trained on 4,719 adult participants from NHANES 2021–2023 using fourteen routine inputs, deliberately excluding creatinine, eGFR and UACR, which are used only to construct the KDIGO-aligned target label. On a held-out test set of 833 participants the cascade attains 70.9% accuracy, 66.0% balanced accuracy and 73.7% recall on the high-risk class, with only two high-to-low misclassifications, the most safety-critical error type. A second contribution converts the numeric risk output into personalised, plain-language education using a retrieval-augmented Qwen3-4B language model fine-tuned with a low-rank adapter and constrained by a deterministic post-generation safety validator. This component was evaluated with a pre-registered six-arm ablation on a frozen 300-profile benchmark that isolates the causal contribution of retrieval, fine-tuning, the validator and prompt compression, analysed with McNemar tests, Wilson intervals and Holm correction. The deployed configuration delivers zero critical-unsafe outputs and 100% emergency-redirect recall, with the validator—not fine-tuning—identified as the component responsible for the safety guarantee.

Keywords: Chronic kidney disease, Risk screening, Two-stage cascade, Retrieval-augmented generation, Large language model safety, Low-rank adaptation.

LIST OF ABBREVIATIONS

ADA	American Diabetes Association
API	Application Programming Interface
AUC	Area Under the Curve
BGE-M3	BAAI General Embedding, Multi-lingual/Multi-granularity model
BUN	Blood Urea Nitrogen
CDC	Centers for Disease Control and Prevention
CI	Confidence Interval
CKD	Chronic Kidney Disease
CV	Cross-Validation
eGFR	Estimated Glomerular Filtration Rate
FAISS	Facebook AI Similarity Search
FK	Flesch–Kincaid (readability grade)
GB	Gradient Boosting
HbA1c	Glycated Haemoglobin
IIT	Institute of Information Technology
JU	Jahangirnagar University
JSONB	Binary JavaScript Object Notation (PostgreSQL type)
KDIGO	Kidney Disease: Improving Global Outcomes
KDOQI	Kidney Disease Outcomes Quality Initiative
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MCC	Matthews Correlation Coefficient
ML	Machine Learning
NF4	4-bit NormalFloat quantisation
NHANES	National Health and Nutrition Examination Survey
NKF	National Kidney Foundation
PEFT	Parameter-Efficient Fine-Tuning
QLoRA	Quantised Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
RF	Random Forest
SBP / DBP	Systolic / Diastolic Blood Pressure
SHAP	SHapley Additive exPlanations
UACR	Urine Albumin-to-Creatinine Ratio

LIST OF NOTATIONS

x	Feature vector of one participant (14 routine inputs)
p_1	Stage-1 output, $P(\text{at risk} \mid x)$
p_2	Stage-2 output, $P(\text{high} \mid \text{at risk}, x)$
τ_1	Stage-1 decision threshold (0.44)
τ_2	Stage-2 decision threshold (0.50)
$\hat{y}$	Predicted screening category (low / moderate / high)
$P(c \mid x)$	Reported probability of class c
ϕ_f	Perturbation contribution of feature f
r_f	Clinical reference value of feature f
m	Probability margin between the top two classes
n_{miss}	Number of missing (imputed) input features
r	LoRA rank (16)
α	LoRA scaling factor (32)
A, B	LoRA low-rank factor matrices
W, W'	Base and adapted weight matrices
k	Number of retrieved guideline excerpts (4)
Δ	Paired risk difference between two evaluation arms
p_{Holm}	Holm-corrected significance level

LIST OF FIGURES

III.1 Layered architecture of the proposed system. The screening layer decides the risk category; the education layer may only explain it. Every generated output passes the deterministic validator before delivery. 16

III.2 Two-stage cascade. Stage 1 gates on the probability of being at risk; only gated cases reach Stage 2, which separates high from moderate. 17

III.3 Layered safety validator. Severe violations replace the output with a safe template; moderate violations are repaired in place; the structural check triggers a single regeneration attempt. . . . 21

III.4 End-to-end request pipeline for a single screening, from input validation through to the delivered, cited education document. . 22

IV.1 The KDIGO risk grid used to construct the target label. Both axes are laboratory kidney markers; they define what the model predicts and are never supplied to it as inputs. 25
IV.2 Class distribution across splits. The high-risk class represents only 4.5% of the development set, which dominates the modelling strategy. 26
IV.3 Factor-aware retrieval. Priority topics are derived from the patient's own risk-raising factors, guaranteeing per-topic coverage before remaining slots are filled by similarity. 33
IV.4 Prompt anatomy. The system turn carries the constraints; the user turn carries everything patient-specific. The compact variant is selected automatically when a fine-tuned adapter is active. 34
IV.5 Training-data generation. Targets are produced under the full prompt but paired with the compact prompt, making the fine-tuning objective one of prompt compression. 36
IV.6 LoRA fine-tuning versus serving. The frozen base is held in bfloat16 in both phases; only the low-rank matrices are trained, and the saved adapter is reattached unchanged at serving. . . . 37
IV.7 Fine-tuning loss. Validation loss decreases monotonically across all three epochs, with no divergence between training and validation. 38

VI.1 Confusion matrix of the deployed cascade on the held-out test set. 49
VI.2 Per-class precision, recall and F1. Recall is deliberately favoured over precision for the at-risk classes. 50
VI.3 Composition of the frozen 300-profile evaluation benchmark. . . 52
VI.4 The six-arm ablation. Each pre-specified comparison isolates a single component. 52
VI.5 Critical-unsafe rate before and after validation. Arms A, B and D have no validator, so raw equals delivered by construction. . . 54
VI.6 Emergency-redirect recall on the 60-profile emergency subgroup, with Wilson 95% intervals. 54
VI.7 Forest plot of the pre-specified comparisons. Green intervals are Holm-significant; grey are not. 56
VI.8 Quality and grounding by arm. 57

VI.9 Token cost and latency by arm. The deployed configuration reduces prompt tokens by 25% relative to the full-prompt fine-tuned arm. 59

LIST OF TABLES

II.1 Representative machine-learning work on CKD compared with this thesis. 10

IV.1 Dataset splits and class distribution (NHANES 2021–2023). . . . 25
IV.2 The fourteen deployed model inputs, with stage membership. Stage-1 order reflects the selection ranking within that stage. . . 27
IV.3 Final model configuration of the deployed cascade. 29
IV.4 Knowledge-base sources. All are scoped to prevention, lifestyle and follow-up content. 31
IV.5 LoRA fine-tuning configuration and result. 38

V.1 Technology stack with rationale and trade-offs. 42
V.2 Experimental environment for fine-tuning, evaluation and serving. 43
V.3 HTTP API surface. 44
V.4 The `screenings` table. 46
V.5 Deployment modes. 46

VI.1 Screening cascade performance on the held-out test set ($n = 833$). 48
VI.2 Per-class performance on the held-out test set. 49
VI.3 Comparison with published CKD classification work. The accuracy figures are not directly comparable because the tasks differ. 51
VI.4 Evaluation arms and the contrast each comparison isolates. . . . 53
VI.5 Primary safety outcomes by arm ($n = 300$), with Wilson 95% intervals in brackets. 54
VI.6 Validator disposition for the validated arms. 55
VI.7 Pre-specified paired comparisons (McNemar, Holm-corrected within family). Bold rows are significant after correction. 56
VI.8 Response quality and grounding by arm ($n = 300$). 57
VI.9 Mean token usage and latency per request. 59
VI.10 Selected delivered violation categories (% of 300 profiles; missed red flag is over the 60 emergency profiles). 60

A.1 Input feature schema with validation ranges and explanation reference values. 77

C.1 Validator layers, severity and disposition. 80

TABLE OF CONTENTS

DECLARATION ii

CERTIFICATE iii

ACKNOWLEDGEMENTS iv

ABSTRACT v

LIST OF ABBREVIATIONS vi

LIST OF NOTATIONS vii

LIST OF FIGURES ix

LIST OF TABLES xi

I Introduction 1

I.1 Background . . . 1

I.1.1 How CKD is confirmed clinically . . . 1

I.1.2 What a routine checkup already provides . . . 2

I.1.3 Language models as a patient-education layer . . . 2

I.2 Overview . . . 2

I.3 Problem Statement . . . 3

I.4 Motivation . . . 4

I.5 Objectives . . . 4

I.6 Scope, Assumptions and Limitations . . . 5

I.7 Contributions . . . 6

I.8 Research Outline . . . 7

II Literature Review 8

II.1 Clinical Background: Defining Kidney Risk . . . 8

II.1.1 The KDIGO risk grid . . . 8

II.1.2 Modifiable risk factors . . . 8

II.2 Machine Learning for CKD Prediction . . . 9

II.2.1 The high-accuracy literature and its input dependency . . . 9
II.2.2 Prediction models for kidney-failure progression . 9
II.2.3 Positioning of the present work . . . 10
II.3 Learning under Class Imbalance . . . 10
II.4 Cascaded and Hierarchical Classification . . . 11
II.5 Explainability of Individual Predictions . . . 11
II.6 Large Language Models in Healthcare . . . 11
II.7 Retrieval-Augmented Generation . . . 12
II.8 Parameter-Efficient Fine-Tuning . . . 12
II.9 Safety Guardrails for Generative Systems . . . 13
II.10 Research Gap . . . 14

III System Model 15
III.1 Design Principles . . . 15
III.2 High-Level Architecture . . . 15
III.3 Formal Model of the Screening Cascade . . . 17
III.3.1 Stage 1: the at-risk gate . . . 17
III.3.2 Stage 2: the high-risk discriminator . . . 18
III.3.3 Composition . . . 18
III.4 Local Explanation Model . . . 19
III.5 Reliability Model . . . 19
III.6 Education Layer Model . . . 20
III.6.1 Retrieval operator . . . 20
III.6.2 Safety validator model . . . 21
III.7 End-to-End Request Algorithm . . . 22

IV Methodology 24
IV.1 Dataset . . . 24
IV.1.1 Source . . . 24
IV.1.2 Target label construction . . . 24
IV.1.3 Splits and class distribution . . . 25
IV.2 Feature Set . . . 26
IV.2.1 Selection . . . 26
IV.2.2 Preprocessing and missing values . . . 27
IV.3 Handling Class Imbalance . . . 28
IV.4 Model Development . . . 28
IV.4.1 Algorithms and hyperparameters . . . 28
IV.4.2 Validation protocol . . . 29

IV.4.3 Deployment refit 29
IV.4.4 Threshold selection 29
IV.5 Explanation Reference Values 30
IV.6 Knowledge Base Construction 30
IV.6.1 Source selection and scoping 30
IV.6.2 Chunking and annotation 31
IV.7 Retrieval 32
IV.7.1 Embedding and indexing 32
IV.7.2 Factor-aware selection 32
IV.8 Prompt Engineering 33
IV.8.1 The full system prompt 34
IV.8.2 The compact system prompt 35
IV.8.3 The user turn 35
IV.9 Training Data Generation 35
IV.10 Parameter-Efficient Fine-Tuning 36
IV.11 Safety Validator Implementation 39
IV.12 Citation Layer 39

V Implementation 41
V.1 Technology Stack 41
V.1.1 A note on the embedding runtime 42
V.2 Experimental Environment 43
V.3 Backend Service 44
V.4 Web Interfaces 44
V.5 Research Database 45
V.6 Deployment Modes 46
V.7 Reproducibility 47

VI Results and Evaluation 48
VI.1 Screening Cascade Results 48
VI.1.1 Evaluation protocol 48
VI.1.2 Headline metrics 48
VI.1.3 Confusion structure 49
VI.1.4 Comparison with the literature 50
VI.2 Language-Model Evaluation: Design 51
VI.2.1 Benchmark 51
VI.2.2 Six-arm ablation 52
VI.2.3 Outcome measures and statistical analysis 53
VI.3 Primary Safety Results 53

VI.4 Pre-Specified Comparisons . . . 55
VI.5 Quality, Grounding and Retrieval . . . 57
VI.6 Efficiency . . . 58
VI.7 Unsafe-Category Breakdown . . . 59
VI.8 Corrections and Audit Trail . . . 60
VI.9 Summary of Findings . . . 61

VII Discussion 62
VII.1 Interpreting the Screening Performance . . . 62
VII.1.1 What 70.9% accuracy means here . . . 62
VII.1.2 Was the cascade justified? . . . 62
VII.1.3 The moderate class . . . 63
VII.2 What Actually Delivers Safety . . . 63
VII.3 The Role of Fine-Tuning . . . 64
VII.3.1 The prompt–model matching effect . . . 64
VII.4 Personalisation . . . 65
VII.5 Threats to Validity . . . 65
VII.5.1 Internal validity . . . 65
VII.5.2 External validity . . . 66
VII.5.3 Statistical validity . . . 66
VII.5.4 Construct validity . . . 66
VII.6 Ethical Considerations . . . 67
VII.7 Deployment Considerations . . . 67

VIII Conclusion and Future Work 69
VIII.1 Conclusion . . . 69
VIII.2 Future Work . . . 70
VIII.3 Closing Remarks . . . 71

References 73

A Input Feature Schema 77

B Safety Rules and Output Contract 78

C Validator Detection Layers 80

D Worked Example 81

E Reproduction Instructions 82

CHAPTER I

Introduction

I.1 Background

Chronic kidney disease (CKD) is a progressive, largely irreversible loss of kidney function that affects an estimated 800 million people worldwide, or roughly one adult in ten [6, 1]. It is a leading cause of cardiovascular mortality, and in its terminal stage it requires dialysis or transplantation—interventions that are financially and logistically out of reach for a large proportion of the population in low- and middle-income countries such as Bangladesh.

The clinical tragedy of CKD is that it is both *slow* and *silent*. The disease typically develops over years, and a person can lose a substantial fraction of kidney function before experiencing any symptom that would prompt a medical visit. By the time symptoms such as oedema, persistent fatigue, or reduced urine output appear, the disease has often reached a stage at which the remaining clinical options are limited to slowing further decline rather than restoring function. Early identification is therefore the single most effective lever available, because the dominant drivers of CKD progression—elevated blood pressure, poorly controlled blood glucose, and central obesity—are precisely the factors that respond to early lifestyle modification and routine clinical follow-up.

I.1.1 How CKD is confirmed clinically

The internationally accepted framework for staging CKD is the KDIGO risk grid [1], which combines two laboratory measurements:

- the **estimated glomerular filtration rate** (eGFR), computed from serum creatinine, which measures how effectively the kidneys filter blood; and
- the **urine albumin-to-creatinine ratio** (UACR), which measures how much protein is leaking into the urine.

Together these two axes define the low, moderate, high and very-high risk categories shown later in Figure IV.1. This grid is the diagnostic gold standard, and nothing in this thesis proposes to replace it.

The practical difficulty is one of *ordering*, not of accuracy. Neither eGFR nor UACR is part of a routine general health checkup. A creatinine assay with an eGFR calculation, and particularly a urine albumin test, are kidney-specific orders that a clinician places when they already have a reason to suspect kidney

disease. For an asymptomatic patient there is usually no such trigger. The result is a circular detection gap: the test that would reveal early CKD is not ordered because there is no symptom, and there is no symptom because the disease is still early. This gap—not the cost of a blood draw as such—is the specific problem this research addresses.

I.1.2 What a routine checkup already provides

A standard general health checkup, by contrast, routinely produces a substantial amount of information that is physiologically related to kidney health: blood pressure, waist circumference, fasting glucose, glycated haemoglobin (HbA1c), blood urea nitrogen (BUN), uric acid, serum albumin and haemoglobin, together with a self-reported history of diabetes and hypertension. These measurements are inexpensive, widely available, and— critically—*already being collected* in general practice, community health camps and population health surveys. If a sufficiently reliable risk signal can be extracted from measurements that already exist, then screening becomes an act of computation rather than an act of additional testing.

I.1.3 Language models as a patient-education layer

A risk score on its own has limited value to the person being screened. Being told "your kidney risk is moderate" answers neither "what does that mean for me?" nor "what should I do next?". Health literacy about kidney disease is low in the general population, and the gap between a numeric model output and an actionable understanding is precisely where most screening tools stop.

Recent instruction-following large language models (LLMs) can produce fluent, readable explanations, and are therefore an attractive candidate for closing that gap. They are also, however, unreliable in exactly the ways that matter most in a medical context: they can state a diagnosis that the underlying model never made, recommend or adjust medication, invent numeric dietary targets, or reassure a patient whose reported vital signs indicate an emergency. Deploying an LLM in a health-screening pathway is therefore not primarily a generation problem but a *constraint* problem.

I.2 Overview

This thesis presents **Nephro-AI**, an end-to-end system that pairs a cross-sectional machine-learning kidney-risk screening model with a safety-constrained, guideline-grounded language-model education layer. The system is built around two design commitments that run through every chapter that follows.

1. **The screening model consumes only routine, low-cost inputs.** Creatinine, eGFR, UACR and urine protein are excluded from the feature set entirely. They are used exclusively to construct the training label from the KDIGO grid, and never as model inputs. The deployed model therefore runs on data a general checkup already produces.
2. **Safety is enforced deterministically, not merely requested.** The language model is instructed to behave safely, is fine-tuned on safety-screened examples, and—decisively—has every one of its outputs passed through a rule-based validator that blocks diagnosis, medication and supplement advice, rewrites numeric dietary targets, and injects urgent-care redirection when the reported vitals demand it.

The complete system is implemented as a working web application with a patient-facing interface and a researcher-facing audit interface, backed by a FastAPI service, a PostgreSQL research database, and a local language-model runtime. Both the screening model and the education layer are evaluated quantitatively, the latter through a pre-registered six-arm ablation designed to attribute the observed safety behaviour to specific architectural components rather than to the system as an undifferentiated whole.

I.3 Problem Statement

The problem addressed by this research can be stated as follows.

> *Given only the routine, low-cost measurements produced by a standard general health checkup—and explicitly excluding the kidney-specific confirmatory tests (eGFR, creatinine, UACR)—can a machine-learning system stratify an adult into a clinically meaningful kidney-risk category with sufficient reliability to justify referral for confirmatory testing; and can the resulting risk assessment be communicated to that person as personalised, guideline-grounded education by a large language model whose output is guaranteed, by construction, never to diagnose, prescribe, or withhold urgent care?*

Two sub-problems follow. The first is a supervised learning problem under severe class imbalance: in the population data used here, 82.9% of participants fall in the low-risk category and only 4.5% in the high-risk category, so a classifier optimised for aggregate accuracy will systematically under-detect exactly the group that most needs detection. The second is a constrained natural-language generation problem: the education text must be simultaneously personalised

(referring to the individual's actual measurements), grounded (traceable to published clinical guidance), readable (approximately eighth-grade level), and unconditionally safe.

I.4 Motivation

Three observations motivated this work.

First, the existing machine-learning literature on CKD is methodologically circular for screening purposes. A large body of published work reports very high accuracy—frequently above 95%—for CKD classification [14, 13, 15]. On inspection, these models almost universally include serum creatinine, blood urea, or eGFR among their input features, and are frequently evaluated on a small, clean, well-balanced benchmark dataset. A model that consumes creatinine to predict a creatinine-derived label is performing a computation, not a screening: it requires the very test whose ordering is the bottleneck. The reported accuracy is real but the clinical utility for early detection is limited.

Second, screening tools stop at the score. Systematic review of kidney prediction models [12] finds that the overwhelming majority terminate at a risk estimate, with no patient-facing communication component and no evaluation of whether the information conveyed is understood or actionable. The educational half of the problem is treated as out of scope.

Third, health-domain LLM systems rarely publish measured safety guarantees. Where language models are deployed for patient-facing health information, safety is typically pursued through prompt engineering alone, which offers no guarantee, or through fine-tuning, whose effect is difficult to verify. Published, component-attributed, statistically tested measurements of unsafe output rates are uncommon. Since safety is the property on which deployment permission ultimately depends, it deserves the same evaluative rigour normally reserved for accuracy.

I.5 Objectives

The overall goal of this research is to design, implement and evaluate a kidney-risk screening and education system that is deployable on routinely available data and demonstrably safe in its patient-facing output. The specific objectives are:

1. To construct a KDIGO-aligned, cross-sectional kidney-risk dataset from NHANES 2021–2023 in which the target label is derived from eGFR and UACR while the feature set contains neither.

2. To design and train a two-stage classification cascade that is explicitly optimised to minimise the clinically most dangerous error— classifying a genuinely high-risk person as low risk—rather than to maximise aggregate accuracy.
3. To produce a per-patient explanation of each prediction and a transparent reliability indicator, so that the screening output is interpretable rather than opaque.
4. To construct a safety-scoped knowledge base from published clinical guidance (KDIGO, ADA, NKF/KDOQI, CDC) and a retrieval mechanism that selects guidance according to the individual's own dominant risk factors.
5. To adapt a compact open-weight language model, by parameter-efficient fine-tuning, to produce structured, personalised, guideline-grounded education from a short prompt.
6. To implement a deterministic post-generation safety validator that enforces the clinical constraints unconditionally, independently of what the language model produces.
7. To evaluate the education layer with a pre-registered ablation that attributes the measured safety behaviour to individual components, using appropriate paired statistical inference.
8. To deliver the whole system as a working, auditable web application.

This work also seeks to answer the following research questions:

1. How much kidney-risk discrimination is recoverable from routine, low-cost inputs alone, once the confirmatory markers are withheld?
2. Does decomposing the three-class problem into a cascade of two binary decisions reduce the rate of high-to-low misclassification relative to a single multi-class classifier?
3. Which architectural component—retrieval, fine-tuning, or the deterministic validator—is actually responsible for the safety of the delivered patient education?
4. Can a fine-tuned model reproduce full-prompt safety behaviour from a substantially shorter prompt, and at what measurable cost?

I.6 Scope, Assumptions and Limitations

The following assumptions delimit the work.

- **Screening, not diagnosis.** The system outputs a risk category intended to trigger confirmatory testing. It does not diagnose CKD, and every patient-

facing surface states this explicitly.

- **Cross-sectional design.** Both the training data and each individual screening are point-in-time. NHANES provides a single observation per participant, so the model cannot represent trajectory, and the system does not track change across visits.
- **Label fidelity.** The target variable is a KDIGO-grid category computed from measured eGFR and UACR within the survey. It is a laboratory-derived risk stratification, not an adjudicated clinical diagnosis with imaging or biopsy confirmation.
- **Population.** The model is trained on a United States adult survey population. Transfer to a Bangladeshi or other South Asian population is untested and would require local validation.
- **Language-model evaluation is synthetic.** The 300-profile benchmark used to evaluate the education layer is generated and scored against the deployed cascade, not drawn from real patients, and no clinician review of the generated text was conducted in this phase.
- **Rule-based safety detection.** Both the deployed validator and the evaluation detectors are deterministic rule systems. They provide reproducibility and auditability, but cannot be assumed complete against an unbounded space of unsafe phrasings.

I.7 Contributions

The principal contributions of this thesis are:

1. **A routine-input kidney-risk cascade.** A two-stage random-forest/gradient-boosting cascade trained on 4,719 NHANES participants using fourteen routine inputs, achieving 70.9% accuracy and 73.7% high-risk recall on a held-out test set of 833 participants with only two high-to-low misclassifications, without access to creatinine, eGFR or UACR.
2. **Factor-aware retrieval for per-patient grounding.** A retrieval strategy that derives a priority topic ordering from the individual's own model-attributed risk factors and guarantees topic coverage, so that two people in the same risk category receive different guideline grounding.
3. **A prompt-compression fine-tune.** A LoRA adapter trained on 540 safety-screened examples in which targets generated under a long instruction prompt are paired with a short prompt, reducing prompt cost by 25% at serving time without a measurable safety penalty.
4. **A component-attributed safety evaluation.** A pre-registered six-arm

ablation over a frozen 300-profile benchmark, analysed with McNemar tests, Wilson intervals and Holm correction, which isolates the deterministic validator—rather than fine-tuning—as the component responsible for eliminating critical-unsafe output.

5. **A complete, auditable implementation**, including a patient-facing interface, a researcher-facing audit interface exposing retrieval and validator decisions, and a reproducible artefact trail.

I.8 Research Outline

The remainder of this report is structured as follows. **Chapter II** reviews the clinical background of CKD staging, the machine-learning literature on CKD prediction, techniques for learning under class imbalance, model explainability, and the use of large language models, retrieval augmentation, parameter-efficient fine-tuning and safety guardrails in healthcare, closing with a statement of the research gap. **Chapter III** presents the system model: the layered architecture, the formal definition of the cascade, the explanation and reliability formulations, and the end-to-end request algorithm. **Chapter IV** details the methodology—dataset construction and label derivation, feature selection, imbalance handling, model training and threshold selection, knowledge-base construction, retrieval, prompt engineering, training-data generation, fine-tuning, and the safety validator. **Chapter V** describes the implementation: the technology stack with its trade-offs, the API surface, the web interfaces, the database schema and the deployment modes. **Chapter VI** reports results for both components, including the six-arm ablation and its statistical analysis. **Chapter VII** interprets these findings, examines threats to validity, and discusses ethical and deployment considerations. **Chapter VIII** concludes and identifies future work.

CHAPTER II

Literature Review

This chapter surveys the four bodies of work on which this thesis draws: the clinical definition of kidney risk, machine-learning approaches to CKD prediction, techniques for learning under class imbalance and for explaining individual predictions, and the use of large language models—with retrieval augmentation, parameter-efficient fine-tuning, and safety guardrails—in patient-facing healthcare applications. It closes with an explicit statement of the gap this research addresses.

II.1 Clinical Background: Defining Kidney Risk

II.1.1 The KDIGO risk grid

The KDIGO 2024 clinical practice guideline [1] defines CKD as abnormalities of kidney structure or function present for more than three months. Risk is stratified on two orthogonal axes. The first is the estimated glomerular filtration rate (eGFR), computed from serum creatinine and expressed in $\mathrm{mL/min/1.73\,m^2}$, which is binned into categories G1 ($\geq$90) through G5 ($<$15). The second is albuminuria, measured by the urine albumin-to-creatinine ratio (UACR) in mg/g, binned as A1 ($<$30), A2 (30–300) and A3 ($>$300). The cross-product of these two axes yields the four risk categories—low, moderately increased, high and very high—shown in Figure IV.1 of Chapter IV.

This grid is the label source for the present work. It is important to be precise about the direction of the dependency: eGFR and UACR define *what the model is trying to predict*, and are deliberately withheld from *what the model is allowed to see*.

II.1.2 Modifiable risk factors

The epidemiological literature and the CDC's public health guidance [6, 7, 8] identify diabetes and hypertension as the two dominant causes of CKD, together accounting for the majority of incident cases, followed by obesity, cardiovascular disease, age, and family history. The ADA/KDIGO consensus on diabetes management in CKD [3] and the KDIGO blood-pressure guideline [2] both emphasise that glycaemic and blood-pressure control materially alter the trajectory of kidney decline. The KDOQI nutrition guideline [5] governs dietary advice and is the source of an important constraint adopted in this work: individualised numeric

targets for protein, potassium, phosphorus or sodium must not be issued to a patient without the involvement of a registered or renal dietitian.

The practical consequence for a screening model is favourable: the dominant causal drivers of CKD—blood pressure, glycaemic markers, and adiposity—are all routinely measured, and none of them require a kidney-specific laboratory order.

II.2 Machine Learning for CKD Prediction

II.2.1 The high-accuracy literature and its input dependency

A substantial body of work applies supervised learning to CKD detection. Chittora et al. [14] compare a range of classifiers on the UCI CKD dataset and report accuracies above 98%. Aljaaf et al. [13] report comparable performance with predictive analytics support, and Yildirim [15] examines multilayer perceptron performance on imbalanced CKD data. These results are internally valid, but two properties limit their applicability to community screening.

First, the feature sets almost universally contain serum creatinine, blood urea, or eGFR itself. Since the CKD label in these datasets is defined by kidney function, a model given creatinine is effectively re-deriving a quantity that the label already encodes. High reported accuracy is then expected, and it does not demonstrate that the disease could have been detected *before* the confirmatory test was ordered.

Second, the widely used UCI CKD dataset contains only 400 records, with a relatively balanced class distribution that does not reflect the roughly 10:1 prevalence imbalance found in an unselected adult population. Models validated only on such data have not been tested against the imbalance that dominates real screening.

II.2.2 Prediction models for kidney-failure progression

A separate and more clinically rigorous line of work predicts *progression* among patients already diagnosed with CKD. Ramspek et al. [12] systematically review such models and find substantial heterogeneity in predictor sets, frequent risk of bias, and limited external validation. The canonical example, the Kidney Failure Risk Equation, requires eGFR and UACR as inputs by design. These models answer a different question from the one posed here: they estimate how fast a known patient will deteriorate, whereas this thesis asks who should be tested in the first place.

II.2.3 Positioning of the present work

Table II.1 summarises the contrast between representative prior work and the approach taken here.

Table II.1: Representative machine-learning work on CKD compared with this thesis.

Work	**Data**	**Input features**	**Screening usable pre-test?**
Chittora et al. [14]	UCI CKD (n=400)	Includes serum creatinine, blood urea, specific gravity, albumin	No — requires the confirmatory assay
Aljaaf et al. [13]	UCI CKD (n=400)	Includes creatinine-derived markers	No
Yildirim [15]	UCI CKD (n=400)	Includes laboratory renal markers	No
Ramspek et al. [12] (review)	Cohorts of diagnosed patients	eGFR and UACR required by design	No — assumes diagnosis
This work	**NHANES 2021–23 (n=5,552)**	**Vitals, routine blood chemistry, comorbidity history; creatinine / eGFR / UACR excluded**	**Yes**

II.3 Learning under Class Imbalance

In an unselected adult population the high-risk class is rare. A classifier trained to minimise overall error can achieve high accuracy by predicting the majority class almost everywhere, at the cost of failing precisely on the minority class of interest. Lemaître et al. [16] provide the standard toolbox for addressing this: random over-sampling of the minority class, under-sampling of the majority, and hybrid schemes such as SMOTE-based synthesis.

Three consequences guide the design in this thesis. First, aggregate accuracy is an inadequate primary metric; balanced accuracy, per-class recall and the confusion matrix are reported instead. Second, resampling should be applied within the training pipeline only, so that evaluation reflects the true class prior. Third—and central to the present design—not all misclassifications are equally harmful: predicting "low" for a truly high-risk person removes the very referral the system exists to generate, whereas predicting "high" for a low-risk person causes

an unnecessary but ultimately corrective confirmatory test. This asymmetry motivates decision thresholds tuned against a specific error type rather than against symmetric loss.

II.4 Cascaded and Hierarchical Classification

Decomposing a multi-class problem into a sequence of binary decisions is a long-standing technique in detection systems, where an early stage is tuned for high sensitivity to pass a manageable candidate set to a more discriminative later stage. The advantage relevant here is that each stage can be optimised, and its operating threshold selected, against a different criterion: the first stage for recall of the at-risk population, the second for separating severity within that population. A single three-class model, by contrast, exposes only one implicit decision boundary structure and offers no natural place to encode the asymmetric cost described above.

II.5 Explainability of Individual Predictions

For a screening tool intended for non-expert users, an unexplained category is of limited value. Two families of local explanation are widely used. SHAP [20] computes Shapley-value attributions with strong theoretical guarantees but requires either a model-specific efficient implementation or substantial sampling; LIME [21] fits a local surrogate model around the instance. A third, simpler family is single-feature perturbation: the value of one feature is replaced by a reference value and the change in predicted probability is recorded.

The present work uses the perturbation approach, applied against the complete deployed cascade rather than a surrogate, and reports it explicitly as such. The trade-off is accepted deliberately: the method is fast enough to run inside a request, requires no additional dependency, and is trivially auditable, but it does not capture interactions between features and is not a Shapley attribution. Chapter IV states the method formally and Chapter VII returns to its limitations.

II.6 Large Language Models in Healthcare

Singhal et al. [28] demonstrated that large language models encode a substantial amount of clinical knowledge, achieving strong performance on medical question-answering benchmarks, while also documenting failure modes in which fluent output is factually wrong. Thirunavukarasu et al. [29] survey clinical applications and identify hallucination, outdated knowledge, and the absence of provenance as

the primary barriers to deployment. The consistent finding across this literature is that fluency and correctness are decoupled: an incorrect statement is delivered with exactly the same confidence as a correct one, which is particularly hazardous for a lay audience with no basis for scepticism. Broader reviews of natural language processing over clinical text [30] reach a similar conclusion regarding the need for explicit provenance.

For patient education specifically, the requirements are narrower than for clinical decision support but no less strict. The output must not diagnose, must not alter medication, must not issue individualised clinical targets, and must escalate appropriately when presented with red-flag findings.

II.7 Retrieval-Augmented Generation

Retrieval-augmented generation [26] conditions a language model on passages retrieved from an external corpus at inference time, rather than relying solely on knowledge stored in the model's parameters. For a clinical education application this offers three concrete advantages: the knowledge source is explicit and citable, it can be updated by re-indexing rather than retraining when guidelines change, and the retrieved passages constrain the model's factual claims.

Retrieval quality depends on the embedding model. BGE-M3 [25] provides multilingual, multi-granularity dense embeddings trained with self-knowledge distillation, and is used in this work to embed both the knowledge-base passages and the per-patient query.

A limitation of naive top-k retrieval, observed directly during development and addressed in Chapter IV, is that in a narrow domain the same small set of generic passages tends to be returned for every query, because all queries are semantically similar. The retrieval strategy adopted here therefore incorporates the patient's own attributed risk factors into the selection process rather than relying on similarity ranking alone.

II.8 Parameter-Efficient Fine-Tuning

Full fine-tuning of a multi-billion-parameter model requires optimiser state and gradients for every weight, which is beyond the memory budget of commonly available hardware and risks degrading the model's general instruction-following ability. Low-Rank Adaptation (LoRA) [23] freezes the pretrained weights and injects a trainable low-rank decomposition into selected projection matrices, so

that the adapted weight becomes

$$W' = W + \frac{\alpha}{r} BA, \tag{II.1}$$

where W is the frozen weight, $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ are the trainable factors, r is the rank and α a scaling constant. Only A and B receive gradients, reducing the trainable parameter count by orders of magnitude.

QLoRA [24] extends this by holding the frozen base weights in 4-bit NormalFloat quantisation during training, further reducing memory while keeping the adapter itself in higher precision. It is worth stating the relationship precisely, because the terminology is frequently conflated: QLoRA is not a different adapter type from LoRA. It is LoRA training performed with a quantised frozen base. The artefact produced is an ordinary LoRA adapter, and it may subsequently be attached to a base model held at any precision. Quantisation is therefore a memory-saving choice rather than a methodological one: where the GPU can hold the full-precision base, as in the present work, standard LoRA in bfloat16 is preferable because it avoids adding quantisation error to the frozen weights. This thesis uses standard LoRA, not QLoRA.

II.9 Safety Guardrails for Generative Systems

Three broad strategies exist for constraining generative output. *Prompt engineering* states the rules in the instruction; it is cheap and flexible but provides no guarantee, since compliance is probabilistic. *Fine-tuning* attempts to internalise the rules in the weights; it improves the prior but remains statistical and is difficult to verify exhaustively. *Post-generation validation* inspects the produced text and blocks, rewrites, or replaces it; it is the only one of the three that can offer a deterministic guarantee for the patterns it covers, at the cost of being unable to anticipate patterns it does not.

An increasingly common fourth option is to use a second language model as a judge. This scales to novel phrasings but reintroduces probabilistic behaviour at exactly the point where a guarantee is wanted, and adds inference cost and latency.

The literature reporting *measured* unsafe-output rates for deployed patient-facing systems—particularly rates attributed to individual architectural components—is sparse. This observation directly motivates the evaluation design in Chapter VI.

II.10 Research Gap

Synthesising the above, four gaps are identified.

1. **Input circularity in CKD screening models.** Existing high-accuracy CKD classifiers consume the confirmatory markers whose unavailability is the actual bottleneck to early detection. There is little work establishing how much risk discrimination survives when those markers are withheld and only routine checkup measurements remain.
2. **Symmetric treatment of asymmetric errors.** Reported CKD models are optimised and reported on aggregate accuracy, without explicit treatment of the high-to-low misclassification as the dominant clinical harm, and without an architecture that provides a natural place to control it.
3. **Risk scores without communication.** Screening models terminate at a numeric output. Personalised, guideline-grounded, readable communication of what that output means—and its evaluation—is largely absent.
4. **Unmeasured, unattributed LLM safety.** Where language models are used for health education, safety is usually asserted through prompt design rather than measured, and where it is measured, the measurement does not isolate *which* component produces it. Consequently there is no evidence base for deciding whether fine-tuning, retrieval, or an explicit guardrail is the mechanism worth investing in.

This thesis addresses gap (1) by constructing a routine-input-only feature set with a KDIGO-derived label; gap (2) through a two-stage cascade with independently tuned thresholds and explicit reporting of the high-to-low error count; gap (3) by building and deploying a grounded education layer; and gap (4) through a pre-registered six-arm ablation that measures and attributes safety behaviour component by component.

CHAPTER III

System Model

This chapter presents the proposed architecture. It first states the design principles, then describes the layered system structure, formalises the screening cascade and its auxiliary outputs, defines the education layer and its safety model, and finally gives the end-to-end request algorithm.

III.1 Design Principles

Five principles govern the architecture and are referred to throughout the remainder of this report.

1. **Input frugality.** The screening model may consume only measurements that a routine general checkup already produces. The confirmatory markers (eGFR, creatinine, UACR, urine protein) are excluded from the feature space and appear only in label construction.
2. **Asymmetric error control.** The architecture must expose an explicit control point for the high-to-low misclassification, which is treated as categorically more harmful than any other confusion.
3. **Separation of prediction from communication.** The language model may explain, contextualise and personalise the screening result, but may never alter the risk category. The category is decided exclusively by the cascade.
4. **Determinism at the safety boundary.** Constraints that protect the patient are enforced by deterministic rules applied after generation, not by trusting the generative model to comply.
5. **Auditability.** Every intermediate artefact—stage probabilities, attributed factors, retrieved passages, validator decisions—is retained and exposed, so that any delivered output can be reconstructed and explained after the fact.

III.2 High-Level Architecture

The system is organised into four layers, shown in Figure III.1: a presentation and input layer, a machine-learning screening layer, a safety-constrained language-model education layer, and a persistence layer.

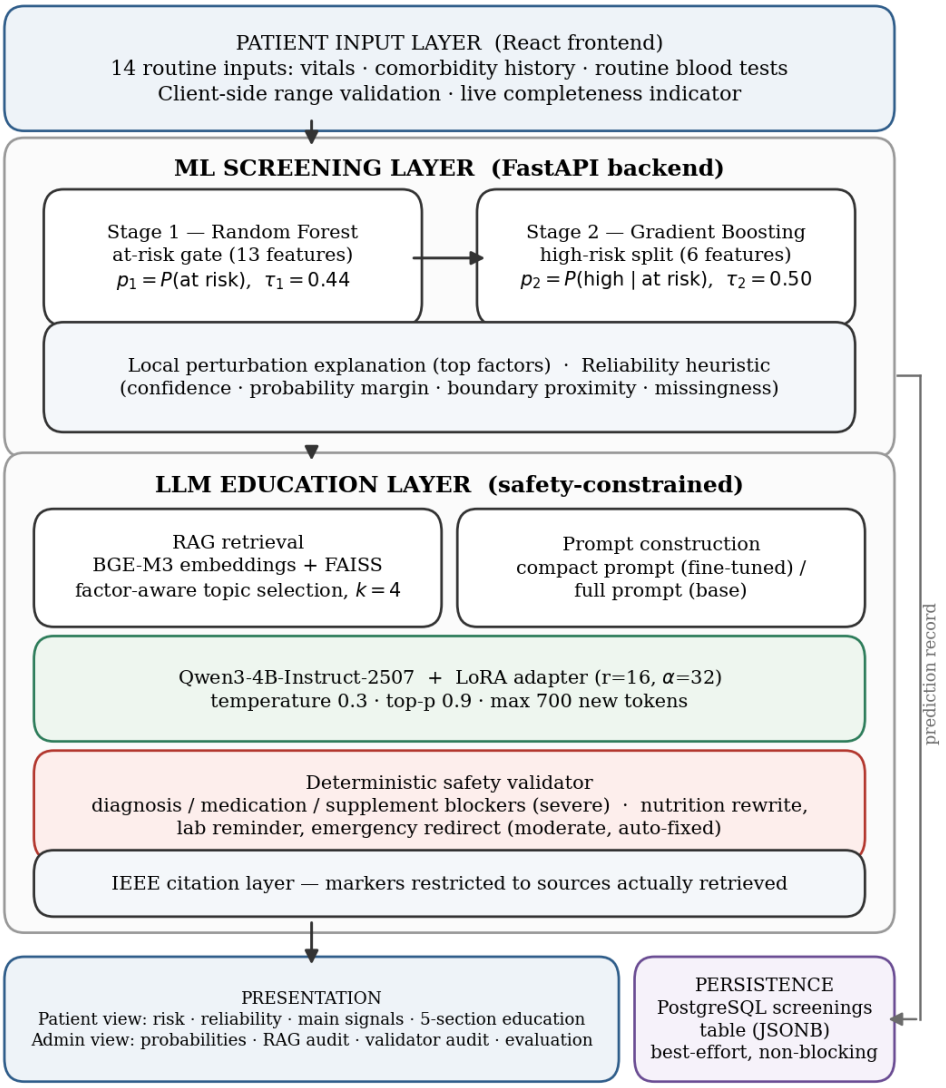


Figure III.1: Layered architecture of the proposed system. The screening layer decides the risk category; the education layer may only explain it. Every generated output passes the deterministic validator before delivery.

The **input layer** collects fourteen routine measurements through a web form with client-side range validation and a live completeness indicator, and submits them to the backend.

The **screening layer** converts the submitted values into a feature vector, applies the two-stage cascade, and produces the risk category, the class probabilities, a per-feature attribution of the decision, and a reliability assessment.

The **education layer** converts that structured output into patient-facing text. It retrieves guideline passages selected according to the individual's attributed risk factors, constructs a prompt containing the screening result, the patient's own values and the retrieved passages, generates five labelled sections with a fine-

tuned language model, and then subjects the result to the deterministic validator and a citation layer.

The **persistence layer** records each screening in a PostgreSQL table for research use. It is deliberately best-effort: a database failure is logged and skipped, never propagated to the user-facing request.

III.3 Formal Model of the Screening Cascade

Let $x \in \mathcal{X}$ denote the feature vector of one individual, where $\mathcal{X}$ is the space of the fourteen routine inputs enumerated in Chapter IV. Let $\mathcal{Y} = \{\text{low}, \text{moderate}, \text{high}\}$ be the screening categories.

Rather than learning a single mapping $\mathcal{X} \to \mathcal{Y}$, the system learns two binary probabilistic classifiers arranged in a cascade, shown in Figure III.2.

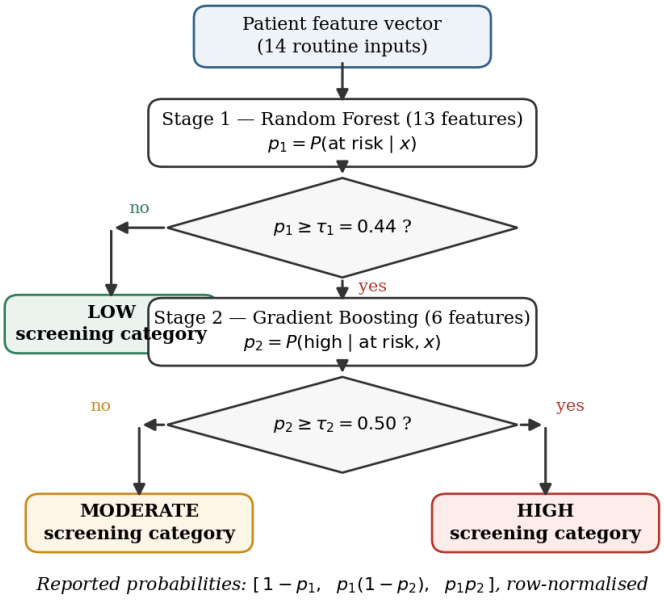


Figure III.2: Two-stage cascade. Stage 1 gates on the probability of being at risk; only gated cases reach Stage 2, which separates high from moderate.

III.3.1 Stage 1: the at-risk gate

Stage 1 is a binary classifier f_1 trained to separate the low category from the union of moderate and high. Writing $S_1 \subset \mathcal{X}$ for the 13-feature subspace used by this stage,

$$p_1 = f_1(x_{S_1}) = P(\text{at risk} \mid x), \qquad \text{at risk} \equiv y \in \{\text{moderate}, \text{high}\}. \tag{III.1}$$

A decision threshold τ_1 gates the cascade:

$$g(x) = \mathbf{1}[p_1 \geq \tau_1], \qquad \tau_1 = 0.44. \tag{III.2}$$

Because τ_1 alone determines whether an individual can ever be assigned a non-low category, it is the architectural control point for the asymmetric error described in principle 2. Lowering τ_1 increases sensitivity to at-risk individuals at the cost of more false referrals; the value 0.44 was selected on development data as described in Section IV.4.4.

III.3.2 Stage 2: the high-risk discriminator

Stage 2 is a second binary classifier f_2, trained on the at-risk subpopulation to separate high from moderate, using a six-feature subspace S_2:

$$p_2 = f_2(x_{S_2}) = P(\text{high} \mid \text{at risk}, x). \tag{III.3}$$

III.3.3 Composition

The predicted category is

$$\hat{y} = \begin{cases} \text{low}, & \text{if } p_1 < \tau_1, \\ \text{high}, & \text{if } p_1 \geq \tau_1 \text{ and } p_2 \geq \tau_2, \\ \text{moderate}, & \text{otherwise}, \end{cases} \qquad \tau_2 = 0.50. \tag{III.4}$$

Applying the chain rule of probability, the three class probabilities reported to the user are

$$\tilde{P}(\text{low} \mid x) = 1 - p_1, \tag{III.5}$$

$$\tilde{P}(\text{moderate} \mid x) = p_1\,(1 - p_2), \tag{III.6}$$

$$\tilde{P}(\text{high} \mid x) = p_1\,p_2, \tag{III.7}$$

normalised so that they sum to unity:

$$P(c \mid x) = \frac{\tilde{P}(c \mid x)}{\sum_{c' \in \mathcal{Y}} \tilde{P}(c' \mid x)}. \tag{III.8}$$

The reported confidence is $\max_c P(c \mid x)$.

Note that p_2 is evaluated for every individual, but enters the decision in Equation III.4 only when the gate is open. This keeps the reported probability vector well defined for all inputs while preserving the gating semantics.

III.4 Local Explanation Model

To make each prediction interpretable, the system attributes the decision to individual inputs by single-feature perturbation against a fixed clinical reference. Let $\hat{y}$ be the predicted category, $P(\hat{y} \mid x)$ the corresponding probability from Equation III.8, and r_f a fixed reference value for feature f. Define $x^{(f \to r_f)}$ as the input x with feature f replaced by r_f and all other features unchanged. The contribution of f is

$$\phi_f = P(\hat{y} \mid x) - P\left(\hat{y} \mid x^{(f \to r_f)}\right). \tag{III.9}$$

A positive ϕ_f means the individual's actual value *supports* the predicted category relative to the reference; a negative value means it pulls away from it. Features are ranked by $|\phi_f|$ and the largest are surfaced.

Two properties should be stated plainly. Each perturbed input is re-scored through the complete cascade, so ϕ_f reflects genuine model behaviour rather than a surrogate approximation. However, because exactly one feature is varied at a time, ϕ_f does not capture interactions: for correlated features the sum of individual contributions need not equal the effect of changing them jointly. The method is therefore reported as model-agnostic local perturbation, not as a Shapley attribution.

III.5 Reliability Model

Not every prediction deserves equal trust. The system computes a deterministic reliability level from three signals. Let $P_{(1)} \geq P_{(2)} \geq P_{(3)}$ be the sorted class probabilities. Define the confidence $c = P_{(1)}$, the margin

$$m = P_{(1)} - P_{(2)}, \tag{III.10}$$

and let n_{miss} be the number of inputs the user left blank, which the fitted preprocessing pipeline imputed. A prediction is considered near the decision boundary when

$$|p_1 - \tau_1| \leq 0.06 \quad \text{or} \quad m < 0.15. \tag{III.11}$$

The reliability level is then

$$\text{rel} = \begin{cases} \text{low}, & n_{\text{miss}} \geq 5 \ \vee \ c < 0.50 \ \vee \ m < 0.08, \\ \text{moderate}, & \text{boundary} \ \vee \ c < 0.65 \ \vee \ n_{\text{miss}} \geq 2, \\ \text{high}, & \text{otherwise}. \end{cases} \tag{III.12}$$

The three conditions encode three distinct failure modes: an input-starved prediction leaning on imputation, a prediction made without conviction, and a prediction that a small perturbation could flip. The rule is deterministic and fully inspectable; it is a transparency device and is explicitly *not* a calibrated uncertainty quantification.

III.6 Education Layer Model

The education layer is a function

$$E : (\hat{y},\, P(\cdot \mid x),\, \{\phi_f\},\, \text{rel},\, x) \longrightarrow \mathcal{S}, \tag{III.13}$$

where $\mathcal{S}$ is the space of five-section structured education documents with the fixed section set

$$\{\text{Summary},\ \text{Food habits},\ \text{Rest and sleep},\ \text{Daily routine},\ \text{Follow-up reminder}\}.$$

E is realised as a composition of four operators:

$$E = V \circ G \circ \Pi \circ R, \tag{III.14}$$

where R is factor-aware retrieval, Π is prompt construction, G is language-model generation, and V is the deterministic safety validator. The ordering matters: V is applied last and unconditionally, so no property established by V can be undone by G.

III.6.1 Retrieval operator

Let $\mathcal{K}$ be the knowledge base of guideline passages, each carrying a topic tag $t(\kappa)$ and a restriction flag. Retrieval builds a query q from the risk category and the highest-ranked attributed factors, embeds it, and returns k passages. Rather than taking the top k by similarity alone, R derives an ordered priority topic list $T(x)$ from the factors that *raise* the individual's risk, and selects the best-scoring passage for each topic in turn before filling remaining slots by score. This guarantees that the retrieved evidence reflects the individual's own dominant risk drivers rather than the generic centroid of the corpus, and is what makes two individuals in the same category receive different grounding.

III.6.2 Safety validator model

The validator is a function $V : \mathcal{T} \times \mathcal{C} \to \mathcal{T} \times \Sigma$ mapping a candidate text and a context to a delivered text and a status $\sigma \in \{\text{pass}, \text{sanitized}, \text{fallback}\}$. It applies an ordered set of detectors, each classified by severity, as shown in Figure III.3.

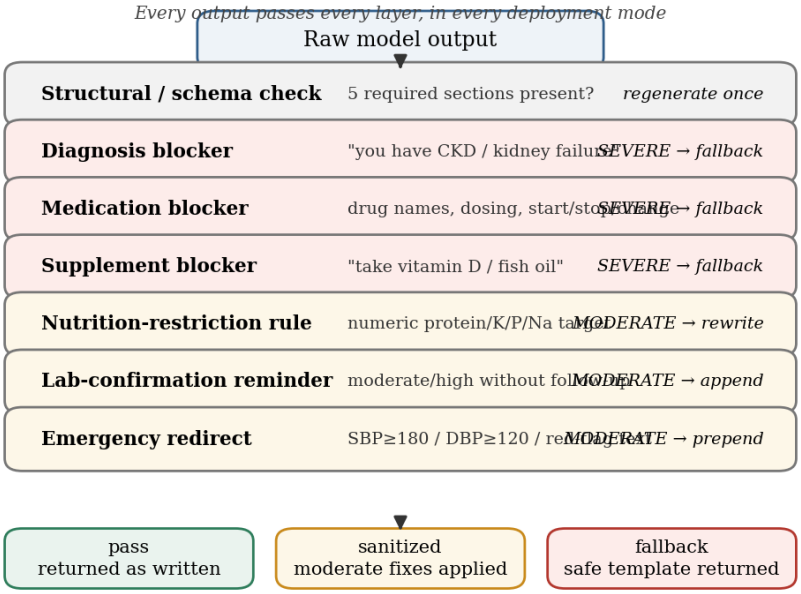


Figure III.3: Layered safety validator. Severe violations replace the output with a safe template; moderate violations are repaired in place; the structural check triggers a single regeneration attempt.

Formally, write w for the candidate text and C for the request context, and let D_{sev} and D_{mod} be the severe and moderate detector families. Then

$$V(w, C) = \begin{cases} (w_{\text{safe}},\ \text{fallback}), & \exists\, d \in D_{\text{sev}} : d(w) = 1, \\ (\rho(w, C),\ \text{sanitized}), & \exists\, d \in D_{\text{mod}} : d(w) = 1, \\ (w,\ \text{pass}), & \text{otherwise}, \end{cases} \tag{III.15}$$

where ρ is the repair operator (rewriting numeric dietary targets into a dietitian referral, appending the laboratory-confirmation reminder, or prepending urgent-care redirection) and w_{safe} is a fixed, condition-aware safe template.

The guarantee this provides is bounded but precise: for every unsafe pattern in $D_{\text{sev}} \cup D_{\text{mod}}$, no text exhibiting that pattern can be delivered, regardless of what the generative model produced. The guarantee does not extend to patterns outside the detector set, a limitation examined in Chapter VII.

III.7 End-to-End Request Algorithm

Algorithm 1 states the complete request path, and Figure III.4 shows it schematically.

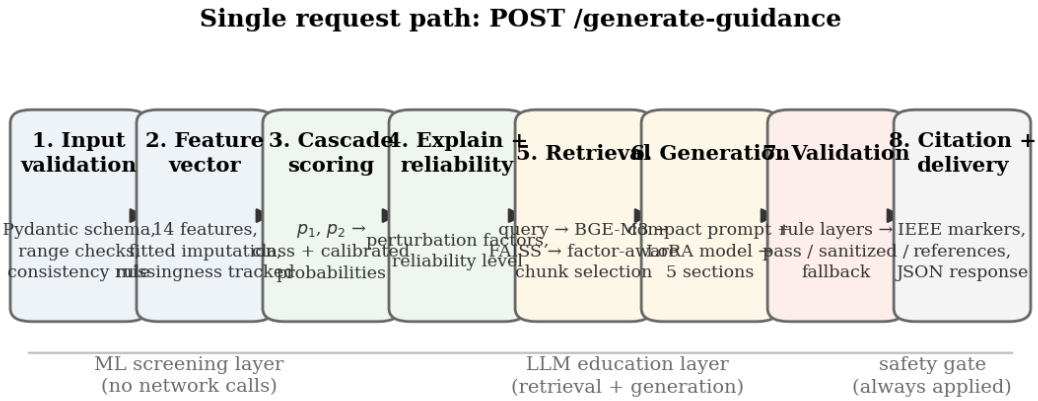


Figure III.4: End-to-end request pipeline for a single screening, from input validation through to the delivered, cited education document.

Algorithm 1 End-to-end kidney-risk screening and education

Require: Submitted measurements u; cascade $(f_1, f_2, \tau_1, \tau_2)$; knowledge base $\mathcal{K}$; language model G; validator V

Ensure: Risk category $\hat{y}$, delivered education document s, status σ

1: $u \leftarrow \textsc{ValidateSchema}(u)$ ▷ reject out-of-range / unknown fields
2: $x \leftarrow \textsc{BuildFeatureVector}(u)$ ▷ consistency rule; missing → NaN
3: $n_{\text{miss}} \leftarrow |\{f : x_f = \text{NaN}\}|$
4: $p_1 \leftarrow f_1(x_{S_1})$; $\quad p_2 \leftarrow f_2(x_{S_2})$ ▷ fitted imputation inside each stage
5: **if** $p_1 < \tau_1$ **then**
6: $\quad \hat{y} \leftarrow$ low
7: **else if** $p_2 \geq \tau_2$ **then**
8: $\quad \hat{y} \leftarrow$ high
9: **else**
10: $\quad \hat{y} \leftarrow$ moderate
11: $P(\cdot \mid x) \leftarrow \textsc{Normalise}(1 - p_1,\ p_1(1 - p_2),\ p_1 p_2)$
12: **for** each supplied feature f with reference r_f **do**
13: $\quad \phi_f \leftarrow P(\hat{y} \mid x) - P(\hat{y} \mid x^{(f \to r_f)})$ ▷ re-scores the full cascade
14: $\Phi \leftarrow$ top features by $|\phi_f|$
15: rel $\leftarrow \textsc{Reliability}(P(\cdot \mid x), p_1, n_{\text{miss}})$
16: $T \leftarrow \textsc{PriorityTopics}(\Phi)$ ▷ risk-raising factors first
17: $q \leftarrow \textsc{BuildQuery}(\hat{y}, \Phi)$
18: $\mathcal{R} \leftarrow \textsc{FactorAwareRetrieve}(q, T, \mathcal{K}, k)$
19: $\pi \leftarrow \textsc{BuildPrompt}(\hat{y}, P(\cdot \mid x), \Phi, \text{rel}, x, \mathcal{R})$
20: $w \leftarrow G(\pi)$
21: **if** $\neg\, \textsc{StructureOK}(w)$ **then**
22: $\quad w \leftarrow G(\pi \,\|\, \text{format reminder})$ ▷ single retry
23: $(s, \sigma) \leftarrow V(w, \textsc{Context}(x, \hat{y}))$
24: $s \leftarrow \textsc{AttachCitations}(s, \mathcal{R})$ ▷ markers restricted to retrieved sources
25: $\textsc{PersistBestEffort}(u, \hat{y}, P, s, \sigma)$
26: **return** $(\hat{y}, s, \sigma)$

Three ordering properties of Algorithm 1 are load-bearing. First, the risk category is fixed at line 10, before any language-model involvement, so generation cannot influence classification. Second, the validator at line 23 is applied to whatever text generation produced, including the retry, so the safety properties hold unconditionally. Third, citations are attached at line 24 *after* validation, drawing only on the passages actually retrieved at line 18, which makes a fabricated citation structurally impossible.

CHAPTER IV

Methodology

This chapter documents how the system was built: how the dataset and its labels were constructed, how the feature set was selected and the cascade trained, how the guideline knowledge base was assembled and indexed, how the prompts were engineered, how the fine-tuning corpus was generated, and how the safety validator was implemented.

IV.1 Dataset

IV.1.1 Source

The screening model is trained on the National Health and Nutrition Examination Survey (NHANES) 2021–2023 cycle, published by the Centers for Disease Control and Prevention and the National Center for Health Statistics [11]. NHANES combines an in-home interview with a standardised physical examination and laboratory panel administered in a mobile examination centre, and is designed to be representative of the non-institutionalised civilian population of the United States. It was selected for three reasons: it is the most recent publicly available cycle; it contains, in a single linked record per participant, both the routine measurements used as model inputs and the kidney markers required to construct the label; and it is a cross-sectional survey of an unselected population, so the class prior it exhibits is representative of a screening setting rather than of a nephrology clinic.

Adult participants with the required measurements available were retained, yielding a modelling cohort of 5,552 participants. Data preparation was performed in a leakage-aware manner: the extraction, cleaning and splitting were completed before any modelling, and the resulting file was hash-verified at training time to guarantee that the audited dataset and the trained artefact correspond.

IV.1.2 Target label construction

The target variable is a three-level screening category derived from the KDIGO risk grid [1] using the participant's measured eGFR and UACR, as shown in Figure IV.1. The KDIGO "very high" cell group is merged into the **high** category, since the operational consequence for a screening tool—urgent referral for confirmatory assessment—is identical.

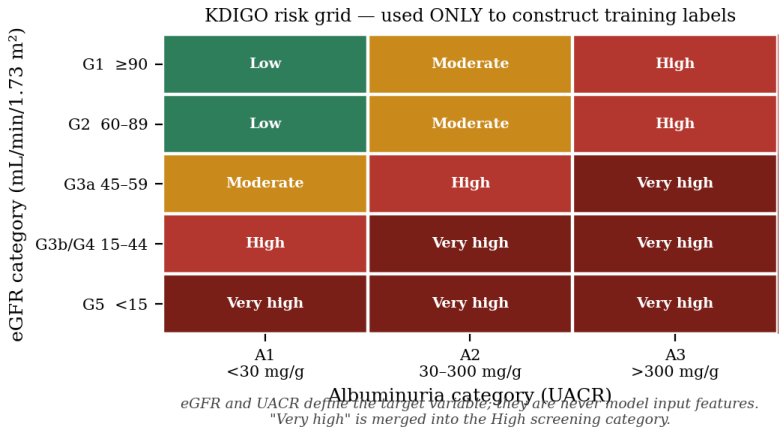


Figure IV.1: The KDIGO risk grid used to construct the target label. Both axes are laboratory kidney markers; they define what the model predicts and are never supplied to it as inputs.

This is the pivotal methodological decision of the ML component and merits emphasis. The markers eGFR and UACR appear *only* on the label side of the learning problem. They are removed from the feature matrix before any model sees the data. The learning task is therefore: predict a laboratory-defined kidney-risk stratification using only measurements that do not include the defining laboratory tests.

IV.1.3 Splits and class distribution

The cohort was partitioned into a training set, a validation set used for model selection and threshold tuning, and a test set held out entirely from development. The union of training and validation constitutes the *development* set on which the deployed model is finally refitted. Table IV.1 and Figure IV.2 give the resulting sizes and class composition.

Table IV.1: Dataset splits and class distribution (NHANES 2021–2023).

Split	N	Low	Moderate	High
Train	3,886	3,222	491	173
Validation	833	691	105	37
Development (deployed fit)	4,719	3,913	596	210
Test (held out)	833	690	105	38

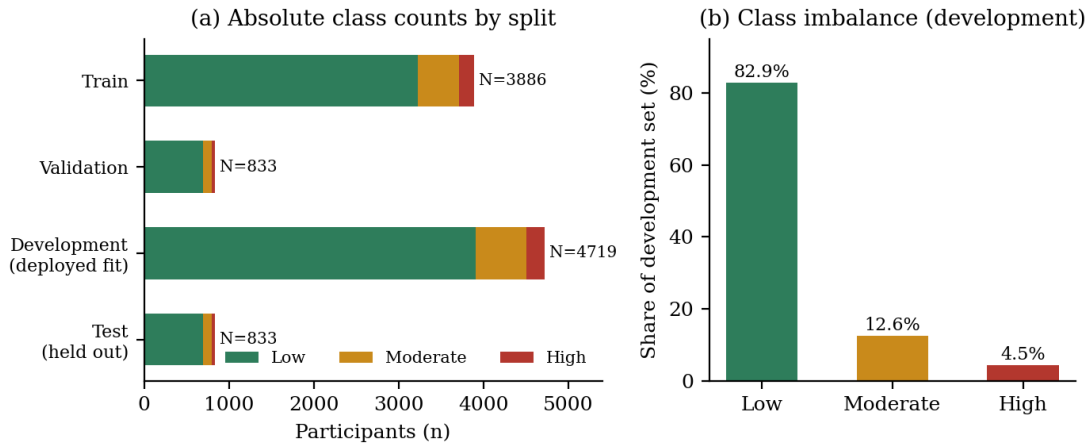


Figure IV.2: Class distribution across splits. The high-risk class represents only 4.5% of the development set, which dominates the modelling strategy.

The imbalance is severe: 82.9% low, 12.6% moderate and 4.5% high in the development set. A trivial classifier predicting "low" for every participant would achieve 82.9% accuracy while detecting nobody, which is why aggregate accuracy is not used as the primary criterion anywhere in this work.

IV.2 Feature Set

IV.2.1 Selection

An initial pool of 22 candidate variables was assembled from the routinely measured NHANES items. Feature selection during model development—driven by cross-validated contribution to each stage—reduced this to fourteen unique model inputs. The variables that did not survive selection into either stage were sex, body mass index, resting pulse, serum sodium, serum potassium, borderline-diabetes status, high-cholesterol history and cholesterol-medication use.

The two stages consume overlapping but distinct subsets: Stage 1 uses thirteen features and Stage 2 uses six, with serum albumin appearing only in Stage 2 and eight variables appearing only in Stage 1. Table IV.2 lists the final set.

Table IV.2: The fourteen deployed model inputs, with stage membership. Stage-1 order reflects the selection ranking within that stage.

Feature	Unit / type	Stage 1	S1 order	Stage 2
Demographic and socioeconomic				
age	years	✓	1	✓
income_poverty_ratio	ratio	✓	10	
Anthropometric and vital signs				
waist	cm	✓	13	
mean_sbp	mmHg	✓	5	✓
mean_dbp	mmHg	✓	11	
Routine blood chemistry				
glucose	mg/dL	✓	6	
hba1c	%	✓	3	
blood_urea_nitrogen	mg/dL	✓	2	✓
uric_acid	mg/dL	✓	8	✓
serum_albumin	g/dL		—	✓
hemoglobin	g/dL	✓	12	✓
Self-reported comorbidity history (binary)				
diabetes_yes	0/1	✓	9	
hypertension_doctor_told	0/1	✓	7	
taking_bp_medicine	0/1	✓	4	

Two observations are worth noting. First, blood urea nitrogen ranks second in Stage 1 and is retained in Stage 2. BUN is a nitrogenous waste product cleared by the kidneys, and is part of a standard basic metabolic panel—unlike creatinine, it is being used here as a routinely available correlate rather than as the creatinine-based marker that defines the label. Second, the Stage-2 subset is dominated by continuous physiological quantities (BUN, uric acid, haemoglobin, serum albumin, systolic pressure, age), consistent with its role of grading severity among individuals already flagged as at risk.

IV.2.2 Preprocessing and missing values

Each stage carries its own fitted `ColumnTransformer`, serialised inside the model package, which applies median imputation to numeric features and most-frequent imputation to binary features, with the imputation statistics learned on the training data only.

One domain-specific consistency rule is applied at inference time before the feature vector is constructed: if a user reports no diagnosed hypertension and

leaves the blood-pressure-medication field blank, that field is set to 0 rather than treated as missing. This mirrors the structural-missingness convention used when the dataset was prepared—a person without a hypertension diagnosis is not on antihypertensive medication—and prevents a distribution shift between training and serving.

The count and identity of every imputed field is retained and returned with the prediction, and feeds directly into the reliability rule of Equation III.12.

IV.3 Handling Class Imbalance

Resampling is applied inside the training pipeline of each stage, never to the validation or test data, so that reported metrics reflect the true class prior.

Stage 1 uses random over-sampling of the minority (at-risk) class to parity (`sampling_strategy` = 1.0). Stage 2 operates on the smaller at-risk subpopulation, where the high/moderate ratio is roughly 1:3, and uses a hybrid over/under-balancing scheme that synthesises minority neighbours ($k = 7$) while removing a fraction (0.25) of majority examples, targeting a 1:1 ratio.

IV.4 Model Development

IV.4.1 Algorithms and hyperparameters

Stage 1 is a random forest [17] and Stage 2 a gradient-boosting classifier [18]. Both are deliberately shallow: the forest is capped at depth 5 and the boosting model at depth 2, with a low learning rate. Given a minority class of only a few hundred examples, constraining capacity was found to matter more than adding it. Table IV.3 lists the final configuration.

Table IV.3: Final model configuration of the deployed cascade.

Component	Parameters
Stage-1 Random Forest	`n_estimators`=150, `max_depth`=5, `max_features`=sqrt, `min_samples_leaf`=2, `min_samples_split`=2, `bootstrap`=true, `max_samples`=1.0
Stage-1 resampler	RandomOverSampler, `sampling_strategy`=1.0
Stage-2 Gradient Boosting	`n_estimators`=150, `learning_rate`=0.05, `max_depth`=2, `min_samples_leaf`=1, `min_samples_split`=2, `subsample`=1.0
Stage-2 resampler	Over/under balancing, `k_neighbors`=7, `majority_remove_frac`=0.25, `target_ratio`=1.0
Cascade thresholds	$\tau_1 = 0.44$, $\tau_2 = 0.50$
Libraries	scikit-learn 1.6.1, imbalanced-learn 0.14.1, NumPy 2.0.2

IV.4.2 Validation protocol

Model selection used repeated stratified cross-validation on the training data with 5 folds and 3 repeats (15 fits per configuration), with a fixed cross-validation seed and a fixed model seed for reproducibility. Interval estimates on the held-out set were obtained by bootstrap resampling with 2,000 replicates, and stability across evaluation seeds was checked over five seeds (42, 52, 62, 72, 82). The complete configuration is captured by a fingerprint recorded with the run, and the deployment package was verified after serialisation by reloading it and confirming that predictions match the in-memory model to within 2.2×10^{-16}.

IV.4.3 Deployment refit

Two model instances are distinguished throughout this report. The *train-only* instance is fitted on the 3,886 training records and is used for development auditing. The *full-development* instance, which is the one deployed and reported as the final result, is refitted on all 4,719 development records (train + validation) with identical hyperparameters and thresholds, after those hyperparameters and thresholds were frozen. The held-out test set of 833 participants was never used in either fitting procedure. All results in Chapter VI refer to the deployed full-development instance unless stated otherwise.

IV.4.4 Threshold selection

With the classifiers fixed, the two thresholds determine the operating point. τ_1 was selected on development data to prioritise sensitivity of the at-risk gate, since

an individual filtered out at Stage 1 can never be assigned a non-low category, making a false negative there unrecoverable. The chosen value of 0.44 sits below the naive 0.50, deliberately widening the gate. τ_2 was left at the neutral 0.50, as the moderate/high distinction carries a far smaller asymmetry: both categories generate a referral, differing only in urgency. Thresholds are stored inside the serialised model package and read at load time, so re-tuning them does not require a code change.

IV.5 Explanation Reference Values

The perturbation explanation of Equation III.9 requires a reference value r_f per feature. These are fixed clinically typical values—age 55 years, systolic 125 mmHg, diastolic 78 mmHg, glucose 100 mg/dL, HbA1c 5.6%, BUN 14 mg/dL, uric acid 5.5 mg/dL, serum albumin 4.2 g/dL, haemoglobin 14 g/dL, waist 98 cm, income-to-poverty ratio 2.0—with all binary comorbidity flags referenced at 0 (condition absent). The interpretation of ϕ_f is therefore "how much does this person's actual value shift the predicted-class probability, relative to a typical value for that measurement". These references are a design choice rather than dataset statistics, and the explanation is reported as relative to them.

IV.6 Knowledge Base Construction

IV.6.1 Source selection and scoping

Ten source documents were collected, spanning professional clinical guidelines [1, 2, 3, 4, 5] and patient-education material [6, 7, 8, 9, 10], listed in Table IV.4. Each was scoped at ingestion to the sections relevant to lifestyle, prevention and follow-up; drug-therapy sections were deliberately excluded from the ingested page ranges, so that pharmacological content is largely absent from the corpus before any runtime safety mechanism is applied.

Table IV.4: Knowledge-base sources. All are scoped to prevention, lifestyle and follow-up content.

ID	Document	Organisation	Year	Restricted
KDIGO2024	Clinical Practice Guideline for the Evaluation and Management of CKD	KDIGO	2024	
KDIGO2021BP	Clinical Practice Guideline for the Management of Blood Pressure in CKD	KDIGO	2021	
ADAKDIGO2022	Diabetes Management in CKD — ADA/KDIGO Consensus Report	ADA / KDIGO	2022	
ADA2026	Standards of Care in Diabetes, Ch. 11: CKD and Risk Management	ADA	2026	
KDOQI2020NUTR	KDOQI Clinical Practice Guideline for Nutrition in CKD	NKF (KDOQI)	2020	✓
NKFPED	Exercise and Chronic Kidney Disease (patient education)	NKF	2026	
CDC_CKD_BASICS	Chronic Kidney Disease Basics	CDC	2024	
CDC_CKD_PREVENT	Preventing Chronic Kidney Disease	CDC	2024	
CDC_CKD_RISK	Risk Factors for Chronic Kidney Disease	CDC	2024	
SYK2015	Save Your Kidneys — Complete Guide for Kidney Patients	Samarpan Kidney Foundation	2015	

IV.6.2 Chunking and annotation

The scoped sources were segmented into 162 passages of approximately 150 words. Each passage carries a stable identifier, a topic tag drawn from {`bp`, `diabetes`, `nutrition`, `weight`, `activity`, `smoking`, `sleep`, `followup`, `hydration`, `prevention`, `general`}, provenance metadata (organisation, title, year, URL) used later for citation, and a safety label of either `safe_public` or `restricted_clinician_referral`.

The restriction flag implements the KDOQI constraint [5]. A passage containing individualised numeric targets for protein, potassium, phosphorus or sodium is flagged rather than deleted. Flagged passages remain retrievable, but are rendered into the prompt with an explicit inline marker instructing the model to refer the reader to a clinician or renal dietitian rather than to repeat the figure. This preserves the clinical context that makes the advice coherent while removing the pathway by which a specific number could reach a patient—and it is backed independently by the validator, which rewrites such numbers if they appear regardless of their origin.

IV.7 Retrieval

IV.7.1 Embedding and indexing

Passages are embedded with BGE-M3 [25], producing 1024-dimensional dense vectors, and indexed with FAISS [27]. The embedder is served through a local Ollama runtime rather than through `sentence-transformers` directly; the reason is documented in Chapter V and concerns a library version constraint rather than model quality. The index records the embedder specification it was built with, and the retriever reconstructs the embedder from that stored specification at query time, so index-time and query-time embeddings can never silently diverge.

IV.7.2 Factor-aware selection

Query construction and selection follow Algorithm 2.

Algorithm 2 Factor-aware retrieval

Require: Attributed factors Φ, risk category $\hat{y}$, knowledge base $\mathcal{K}$, budget k
Ensure: Ordered passage set $\mathcal{R}$, $|\mathcal{R}| \leq k$

1: $q \leftarrow$ risk phrase $\|$ top factor phrases $\|$ mapped clinical terms $\|$ intent anchor
2: $T \leftarrow [\,]$
3: **for** $f \in \Phi$ ordered by $|\phi_f|$, risk-raising factors first **do**
4: **if** topic(f) defined and topic(f) $\notin T$ **then**
5: append topic(f) to T
6: **if** $\hat{y} \in \{\text{moderate}, \text{high}\}$ and `followup` $\notin T$ **then**
7: append `followup` to T
8: $\mathcal{P} \leftarrow \textsc{Search}(\textsc{Embed}(q), \mathcal{K}, k_{\text{pool}})$ ▷ score-ordered pool, $k_{\text{pool}} \geq 24$
9: $\mathcal{R} \leftarrow [\,]$
10: **for** $t \in T$ **do** ▷ one best passage per priority topic, in order
11: **if** $|\mathcal{R}| \geq k$ **then**
12: **break**
13: $\kappa \leftarrow$ first unused $\kappa \in \mathcal{P}$ with topic(κ) $= t$
14: **if** κ exists **then**
15: append κ to $\mathcal{R}$
16: fill remaining slots of $\mathcal{R}$ from $\mathcal{P}$ by descending score
17: re-sort $\mathcal{R}$ by descending score
18: **return** $\mathcal{R}$

Figure IV.3 illustrates the process. The essential property is that line 3–8 make the *selection* depend on the individual, not only the *ranking*. Without this

step, in a corpus this narrow the highest-scoring passages are near-identical across patients, and per-patient grounding degenerates.

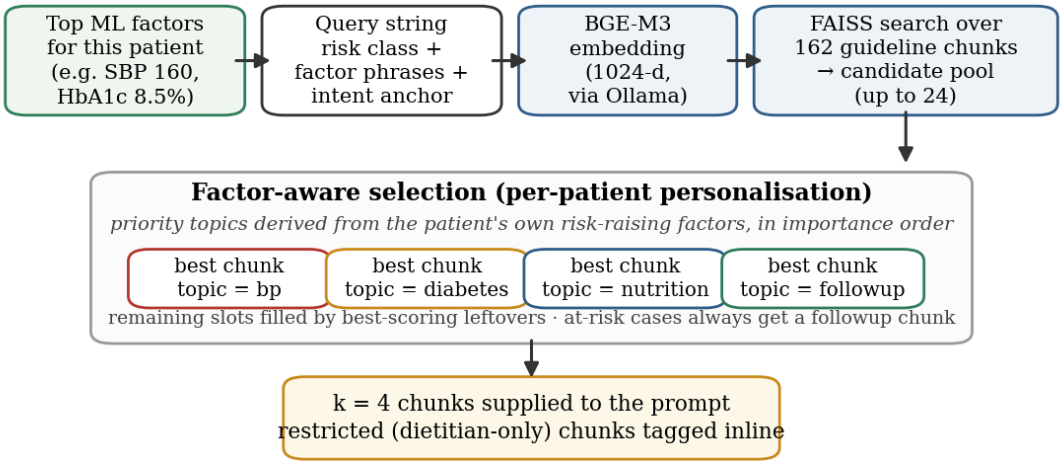


Figure IV.3: Factor-aware retrieval. Priority topics are derived from the patient's own risk-raising factors, guaranteeing per-topic coverage before remaining slots are filled by similarity.

IV.8 Prompt Engineering

Two system-prompt variants exist and exactly one is selected automatically per request, determined by whether a fine-tuned adapter is attached. Figure IV.4 shows the anatomy of both turns.

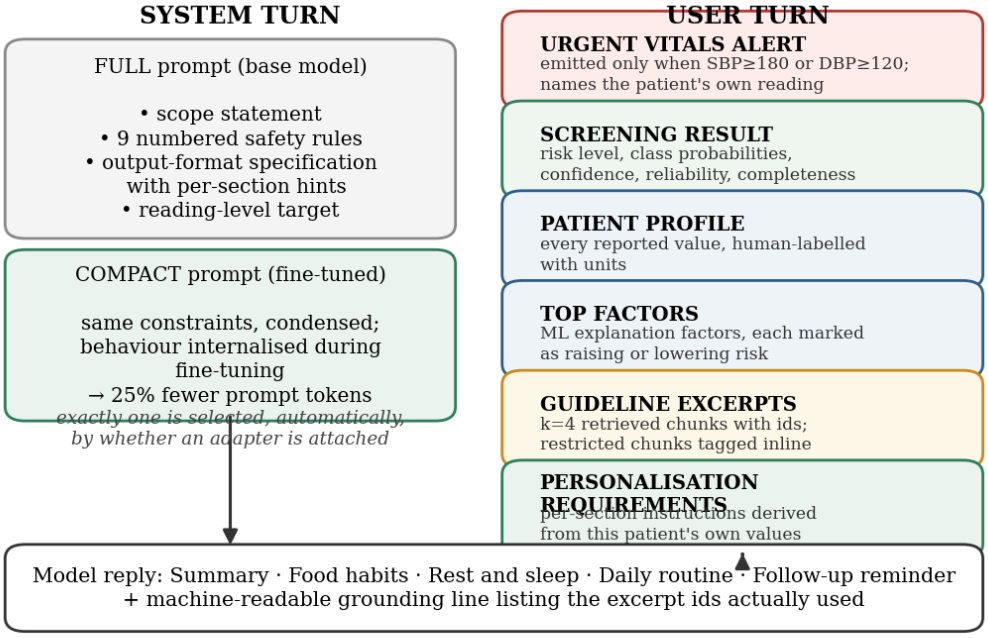


Figure IV.4: Prompt anatomy. The system turn carries the constraints; the user turn carries everything patient-specific. The compact variant is selected automatically when a fine-tuned adapter is active.

IV.8.1 The full system prompt

The full variant states the assistant's scope, enumerates nine safety rules, and specifies the output format literally. The nine rules are: (1) never state or imply a diagnosis; (2) never prescribe, recommend, start, stop, change or dose any medication or supplement; (3) never give individualised numeric targets for protein, potassium, phosphorus or sodium, deferring instead to a clinician or renal dietitian; (4) never position the output as a substitute for clinician judgement, eGFR or UACR testing; (5) *do* tailor wellness education to the individual's stated conditions; (6) always include a clinician and laboratory follow-up reminder for a moderate or high result; (7) advise prompt medical care when the inputs suggest an urgent problem; (8) ground the education in the supplied guideline excerpts; and (9) write at approximately an eighth-grade reading level.

Rule 5 deserves comment because it was revised during development. It originally read "only give general wellness education", which the model followed correctly—producing near-identical generic text for every patient. The instruction was in direct tension with the personalisation objective. The revised rule authorises explicit reference to the individual's conditions while leaving the prohibitions on diagnosis, prescription and numeric targets fully intact.

IV.8.2 The compact system prompt

The compact variant expresses the same constraints and the same output contract far more briefly, on the premise that a model fine-tuned on full-prompt behaviour no longer needs the full statement at inference time. This is the prompt used by the deployed configuration. Its effect on cost is quantified in Chapter VI.

IV.8.3 The user turn

The user turn is identical in structure under both variants and contains six blocks: an urgent-vitals alert (emitted only when systolic ≥ 180 or diastolic ≥ 120 mmHg, and naming the individual's own triggering reading); the screening result with probabilities, confidence and reliability; the complete patient profile with human-readable labels and units; the attributed top factors, each marked as raising or lowering risk; the retrieved guideline excerpts with their identifiers and restriction tags; and a per-section personalisation requirement block.

The personalisation block is computed in Python from the individual's own values using transparent threshold rules—for example, elevated blood pressure is flagged when a hypertension diagnosis is reported or systolic exceeds 130 mmHg—and produces instructions such as "Food habits: focus on low-sugar, low-refined-carbohydrate choices because of elevated blood sugar; suggest reducing salt intake because of high blood pressure". This block exists because the abstract instruction to personalise proved insufficient in early testing: naming the specific fact to reference, and the section to reference it in, was what actually changed the output.

IV.9 Training Data Generation

Fine-tuning requires paired examples. Because no corpus of safety-validated, guideline-grounded kidney education exists, the training corpus was generated by self-distillation through the system's own pipeline, as shown in Figure IV.5.

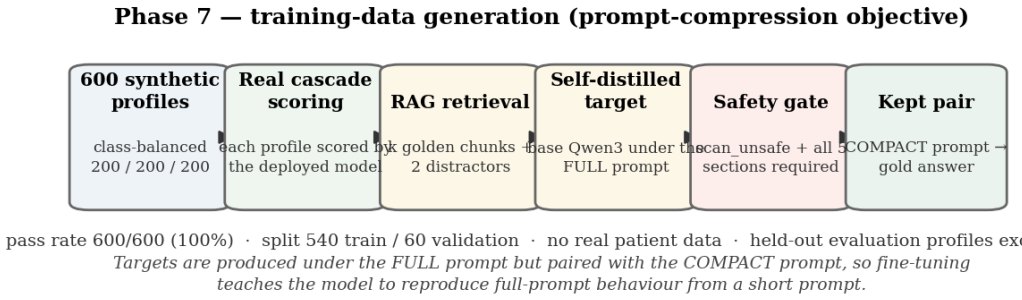


Figure IV.5: Training-data generation. Targets are produced under the full prompt but paired with the compact prompt, making the fine-tuning objective one of prompt compression.

Six hundred synthetic patient profiles were generated, balanced at 200 per risk class. Each profile was scored by the real deployed cascade, so its stated risk category is the one the production model actually assigns. Retrieval was run for each profile with four golden passages plus two distractors, and a target answer was generated by the base Qwen3 model under the *full* prompt. Every candidate target then had to pass an automated gate: it was rejected unless it cleared the unsafe-content scan, contained all five required sections, and mentioned the risk level. All 600 candidates passed, and the corpus was split 540/60 into training and validation.

Two properties of this construction are important. First, the stored example pairs the *compact* prompt with a target generated under the *full* prompt, which makes the learning objective explicitly one of prompt compression—reproducing long-prompt behaviour from a short prompt—rather than of teaching new clinical content. Second, no real patient data is involved at any point, and the held-out evaluation profiles and the ML test set were excluded from the generation pool to prevent leakage into the later evaluation.

IV.10 Parameter-Efficient Fine-Tuning

The base model is Qwen3-4B-Instruct-2507 [22]. The adapter was trained with standard LoRA [23]: the base model's weights were frozen and held in bfloat16, while low-rank adapters were trained on the seven projection matrices of each transformer block. The training script also supports QLoRA [24], which would hold the frozen base in 4-bit NormalFloat; this was deliberately not used. The 24 GB GPU (Section V.2) holds the full bfloat16 base alongside the adapter's optimiser state, so quantisation would have saved memory that was not needed

while adding quantisation error to a small-data run. The adapter trains 33.0 M parameters, 0.81% of the model. The relationship between the training-time and serving-time configurations is shown in Figure IV.6.

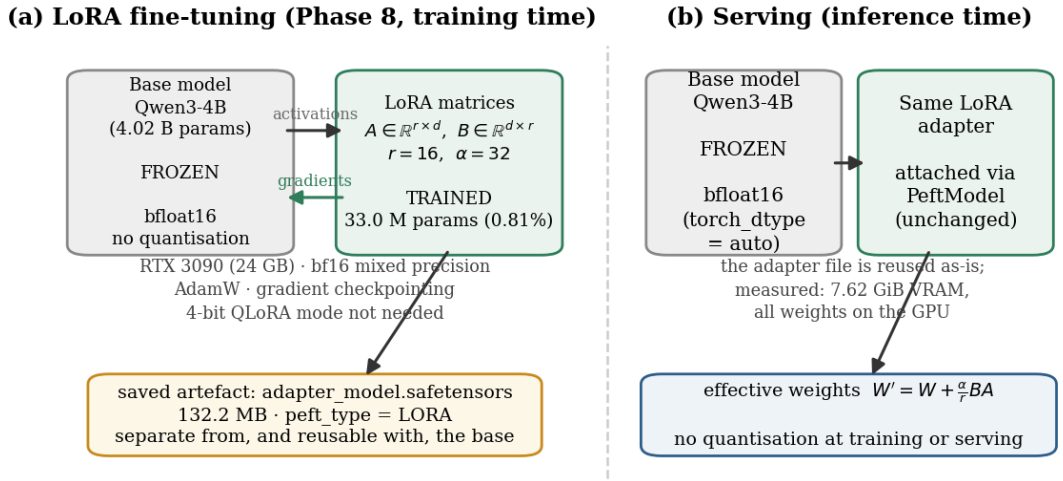


Figure IV.6: LoRA fine-tuning versus serving. The frozen base is held in bfloat16 in both phases; only the low-rank matrices are trained, and the saved adapter is reattached unchanged at serving.

Table IV.5 gives the configuration and outcome. The loss is masked to the assistant's answer tokens only, so the model is not trained to reproduce the prompt.

Table IV.5: LoRA fine-tuning configuration and result.

Setting	**Value**
Base model	Qwen3-4B-Instruct-2507
Adapter	LoRA, $r = 16$, $\alpha = 32$, dropout 0.05
Target modules	`q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj`
Training precision	bfloat16 mixed precision; no quantisation
Optimiser	AdamW (`adamw_torch`); gradient checkpointing
Optimisation	3 epochs, learning rate 2×10^{-4}, cosine schedule, 3% warmup
Trainable parameters	33.0 M of 4.06 B (0.81%)
Hardware	NVIDIA GeForce RTX 3090, 24 GB
Batching	per-device 2 × gradient accumulation 8 = effective 16
Sequence length	3,072 tokens; loss masked to the assistant turn
Data	540 training / 60 validation examples
Seed	42
Final training loss	0.133
Best validation loss	0.155 (trace: 0.197, 0.161, 0.155)
Artefact size	132.2 MB

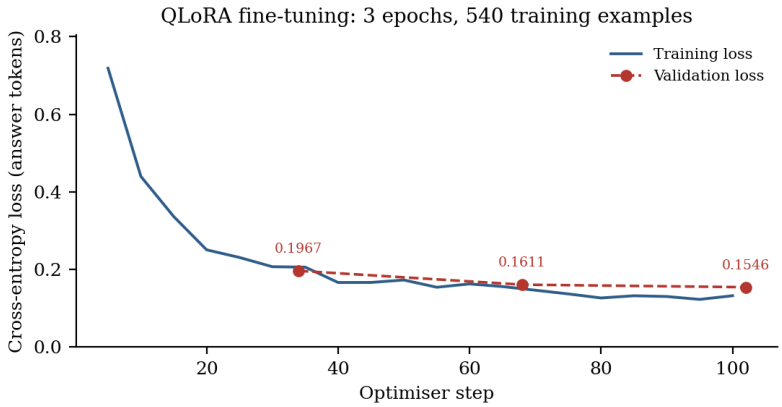


Figure IV.7: Fine-tuning loss. Validation loss decreases monotonically across all three epochs, with no divergence between training and validation.

The monotone validation curve in Figure IV.7 indicates the model had not begun to overfit within three epochs. The absolute loss values are low because the task is highly structured—a fixed five-section format over a constrained vocabulary—so this should be read as evidence of successful format and style internalisation rather than of clinical competence, which is assessed separately in Chapter VI.

IV.11 Safety Validator Implementation

The validator is implemented in pure Python with no machine-learning dependency, so that it runs identically in every deployment mode and can be audited line by line. Detection is sentence-scoped and negation-aware: a sentence is examined in isolation, and a candidate match is discarded if it is governed by a negation or an explicit deferral to a clinician. This distinction matters, because "you should not stop or change any medicine without talking to your doctor" is safe guidance containing medication vocabulary, whereas "stop taking your blood pressure medicine" is a severe violation containing similar vocabulary.

The severe family—diagnosis assertion, medication instruction, supplement recommendation—triggers replacement with a safe template. The moderate family—numeric nutrient targets, missing laboratory-confirmation reminder for an at-risk result, and emergency indications—triggers in-place repair. A structural check verifies that all five sections are present and, if not, triggers a single regeneration before falling back.

Emergency handling deserves specific mention. The validator escalates on measured vital signs (systolic $\geq$ 180 or diastolic $\geq$ 120 mmHg) and on red-flag symptom vocabulary. In the current deployment the input schema exposes no free-text symptom field, so the vitals path is the only one reachable from a live request; the symptom vocabulary is nonetheless implemented, tested, and exercised by the evaluation harness in Chapter VI, and is retained for the schema extension discussed in Chapter VIII. This is stated explicitly rather than left implicit, because the distinction between "implemented and tested" and "reachable in production" is exactly the kind of gap that undermines a safety claim if glossed over.

IV.12 Citation Layer

After validation, a deterministic citation layer attaches IEEE-style markers. Each output section is mapped to the knowledge-base topics it may legitimately draw upon—for example, *Food habits* maps to `nutrition`, `diabetes` and `bp`—and is cited only to sources that were actually retrieved for those topics in this request, refined by the grounding identifiers the model itself reported. Numbering follows first-citation order and the reference list de-duplicates by source document.

Because the mapping is computed from the retrieval record rather than parsed from the generated text, a citation marker cannot point to a source the system did not retrieve. Fabricated references—a well-documented failure mode of generative models, and one a reviewing clinician would immediately catch—are therefore

structurally impossible rather than merely improbable.

CHAPTER V

Implementation

This chapter describes the realised system: the technology stack and the reasoning behind each choice, the service interface, the two web interfaces, the research database, the deployment modes, and the reproducibility measures.

V.1 Technology Stack

Table V.1 records each major technology, where it is used, why it was selected, and its principal drawback. The drawbacks are stated explicitly because several of them constrain claims made later in this report.

Table V.1: Technology stack with rationale and trade-offs.

Technology	Where used	Why chosen	Drawback
scikit-learn [19]	Cascade training and inference	Mature, interpretable, strong on tabular clinical data; fast CPU inference	Lower ceiling than deep tabular methods on very large data
imbalanced-learn	Training-time resampling	Standard implementations of over/under-sampling	Risk of overfitting synthetic minority patterns if not cross-validated
joblib	Model serialisation	Plain-`dict` package with no custom classes; self-describing; service trusts the package's own feature list	Python-specific format
FastAPI [35] + Uvicorn	Backend service	Async, automatic OpenAPI documentation, native Pydantic validation	Blocking ML/LLM calls must be managed carefully in an async context
Pydantic	Request validation	`extra="forbid"` plus per-field ranges reject malformed input before the model sees it	Schema must track the model's feature list
BGE-M3 via Ollama	RAG embedding (1024-d)	Strong open multilingual embedder; serving via Ollama avoids a `torch.load` version restriction	Requires the Ollama service to be running
FAISS	Vector search	Fast, standard, integrates cleanly	Overkill at 162 passages; value is headroom for a larger corpus
Qwen3-4B-Instruct-2507	Text generation	Strong instruction-following at a size that is fine-tunable and self-hostable; no patient data leaves the machine	Quality ceiling below frontier models; needs a GPU for comfortable latency
PEFT	LoRA fine-tuning (bf16)	Trains a 4B model on a single GPU by updating 0.81% of parameters; adapter is small and swappable	Adapter quality bounded by the 540-example corpus
Safety validator (pure Python)	Post-generation gate	Deterministic, auditable, zero inference cost	Cannot cover unseen unsafe phrasings
PostgreSQL + SQLAlchemy + Alembic	Research persistence	JSONB for nested records, typed columns for queries, versioned migrations	An extra service; absence is silent unless health is checked
React + Vite + Tailwind	Web interfaces	Fast builds, small bundle, no heavyweight state library needed	No server-side rendering or routing framework

V.1.1 A note on the embedding runtime

During development the retrieval index was found to have been built with a lexical hashing fallback rather than the intended BGE-M3 embedder. The cause was a library-level security restriction: the installed PyTorch version predates the floor required for a safe `torch.load` path, so the `sentence-transformers` loader raised, and an automatic embedder-selection routine caught the exception and silently degraded to the fallback, emitting only a printed warning.

Two remedies were available: upgrade PyTorch, or serve the same embedding model through a different runtime. The second was chosen, because the language-model serving stack was already verified working against the installed version and an upgrade risked destabilising it for no modelling benefit. BGE-M3 is therefore

served through Ollama, which does not use the restricted loading path. The index was rebuilt, and the retrieval evaluation reported in Chapter VI was re-run entirely against the corrected index.

The episode is documented here rather than omitted because it is methodologically relevant: a silent fallback of this kind degrades results without producing an error, and the corrected and uncorrected runs were both re-scored so that the difference could be attributed.

V.2 Experimental Environment

All language-model work—fine-tuning, the six-arm evaluation of Chapter VI, and live serving—was carried out on a single local workstation, summarised in Table V.2. The software stack was installed before the fine-tuning run and not changed afterwards, so every reported language-model result was produced with identical library versions. All values were read from the machine or from the saved training arguments rather than recalled.

Table V.2: Experimental environment for fine-tuning, evaluation and serving.

Item	**Value**
GPU model	NVIDIA GeForce RTX 3090
VRAM	24 GB (24,576 MiB); driver 560.94
CPU / system memory	Intel Core i9-12900 / 31.8 GB
Operating system	Windows 10 Pro
Python	3.10.11
PyTorch	2.5.1 (CUDA 12.1 build, cuDNN 9.1)
transformers	5.14.1
PEFT / accelerate	0.19.1 / 1.14.0
Training precision	bfloat16 mixed precision, no quantisation
Inference precision	bfloat16 base weights, no quantisation; LoRA weights float32
Inference memory	7.62 GiB VRAM (base model plus adapter)
Decoding (evaluation)	Greedy (deterministic), seed 42
Decoding (live serving)	Sampling, temperature 0.3, top-p 0.9

The inference precision was verified empirically by loading the model exactly as the backend does: every one of the 398 base-weight tensors is bfloat16, the model reports no quantisation configuration, and all parameters reside on the GPU with no CPU offload. The screening cascade, by contrast, requires no GPU; it was trained on Kaggle (Python 3.12, scikit-learn 1.6.1) and is served on the CPU with

the same scikit-learn version. Latency figures reported in Chapter VI are specific to this hardware.

V.3 Backend Service

The backend exposes the endpoints in Table V.3. The model is loaded once at startup; the language-model runtime is a lazily initialised singleton so that a deployment which never requests guidance never pays the model-loading cost.

Table V.3: HTTP API surface.

Method	Endpoint	Purpose
GET	`/health`	Service and model status, library versions, database state
GET	`/db/health`	Database connectivity and stored record count
GET	`/features`	Feature schema, metadata, numeric and binary lists
GET	`/model-info`	Thresholds and per-stage feature lists
POST	`/predict`	Cascade scoring: category, probabilities, explanation, reliability
POST	`/reload-model`	Hot-reload the serialised model package
GET	`/generate-guidance/health`	Language-model mode, model specification, active prompt variant
POST	`/generate-guidance`	Full pipeline: predict, retrieve, generate, validate, cite

Two design details support auditability. First, the model specification string reported by the health endpoint encodes adapter status directly—for example `qwen:Qwen/Qwen3-4B-Instruct-2507+lora:v1/adapter`—so the question "is the fine-tuned model actually serving this request?" is answerable from a single response, and from any stored run record, without inspecting the process that produced it. Second, `/generate-guidance` returns the entire audit trail alongside the patient-facing text: the retrieval query, the retrieved passage identifiers with similarity scores, the raw pre-validator text, the validator status and the list of violations, and the complete prediction object.

V.4 Web Interfaces

The frontend is a single-page React application presenting two distinct interfaces over the same backend.

The **patient interface** presents a fourteen-field form grouped into sections, with per-field normal ranges, inline validation and a live completeness indicator. The result view shows the risk category with a plain statement that it is a screening result and not a diagnosis, the reliability level with an input-completeness bar, a row of "main signals" derived from the top attributed factors expressed in everyday language, and the five education sections with inline citation markers and a reference list.

The **research interface** exposes the full audit trail: both stage probabilities and thresholds, the normalised class probabilities, the input quality panel listing imputed and out-of-range values, the complete factor attribution table with contribution magnitudes, the retrieval panel with the query string and per-passage scores and topics, a side-by-side view of raw and post-validator text, the safety panel showing which validator layers fired, a live system-health panel, and the frozen evaluation metrics.

Exposing the raw pre-validator text next to the delivered text was a deliberate decision. It makes the guardrail's effect directly observable during a demonstration—one can see precisely what the model produced and what was delivered instead—rather than asking an audience to take the safety claim on trust.

V.5 Research Database

Every screening is recorded in a PostgreSQL table, whose schema is given in Table V.4. Nested structures (raw inputs, full prediction, full guidance response) are stored as JSONB so that the schema does not require migration each time a field is added, while the fields most useful for research queries are additionally materialised as typed, indexed columns.

Table V.4: The `screenings` table.

Column	Type	Contents
`id`	INTEGER PK	Row identifier
`created_at`	TIMESTAMPTZ	Time of screening (indexed)
`source`	VARCHAR(32)	`predict` or `generate-guidance` (indexed)
`input_features`	JSONB	Raw submitted values
`n_missing_features`	INTEGER	Count of imputed inputs
`risk_class`	VARCHAR(16)	low / moderate / high (indexed)
`risk_class_display`	VARCHAR(64)	Human-readable category label
`confidence`	FLOAT	Maximum class probability
`probabilities`	JSONB	Full probability vector
`prediction`	JSONB	Complete `/predict` response
`guidance`	JSONB	Complete guidance response, when applicable
`safety_status`	VARCHAR(16)	pass / sanitized / fallback
`llm_mode`	VARCHAR(32)	production / local_demo
`model_spec`	VARCHAR(128)	Model identifier including adapter status

Persistence is best-effort by design. The database layer initialises lazily and, if the connection fails, every helper degrades to a no-op while the API continues to serve screenings normally. A failed write is logged, never surfaced to the user, and never allowed to fail a request. Schema is managed explicitly through Alembic migrations rather than automatic table creation, so that the deployed schema is version-controlled.

V.6 Deployment Modes

Three interchangeable modes are supported behind one interface, selected by environment variable and reported by the health endpoint (Table V.5).

Table V.5: Deployment modes.

	local_demo	**ollama**	**qwen (deployed)**
Language model	Deterministic mock generator	Qwen3-4B via Ollama REST	Qwen3-4B via `transformers` + LoRA adapter
Embedder	Hashing TF-IDF (4096-d)	BGE-M3 (1024-d)	BGE-M3 (1024-d)
Vector search	Brute force	Brute force / FAISS	FAISS
Prompt variant	Full	Full	Compact (adapter active)
GPU required	No	No	Recommended
Safety validator	Identical	Identical	Identical
Purpose	Offline demonstration, CI	Laptop GPU without the ML stack	The evaluated configuration

The offline mode exists so that the complete pipeline—retrieval, prompt construction, validation, citation—can be exercised on any machine without a GPU or network. Its generator is deterministic and is never used for any reported evaluation figure. The safety validator is identical across all three modes, which means the guardrail behaviour can be demonstrated even where the production model cannot run.

V.7 Reproducibility

Several measures support reproduction of the reported results.

- **Dataset integrity.** The prepared modelling file is hash-verified, and the split audit is re-run and asserted at training time.
- **Configuration fingerprinting.** Cross-validation seeds, model seed, bootstrap replicate count, evaluation seeds and thresholds are recorded together with a configuration fingerprint.
- **Serialisation verification.** After the model package is written, it is reloaded and its predictions compared against the in-memory model; agreement was exact to within 2.2×10^{-16}, and the package checksum is recorded.
- **Frozen evaluation benchmark.** The 300-profile language-model benchmark is content-fingerprinted, and the fingerprint was re-verified before and after every generation run to confirm that all arms were evaluated against an identical benchmark.
- **Append-only run records.** Every generated response is stored with the hash of its own raw text, allowing a later re-scoring to prove that only scoring changed and generation did not.
- **Deterministic decoding.** All reported language-model results use a fixed seed with deterministic decoding.

The deployed model package is 302 KB, which together with the 132 MB adapter and the 162-passage index constitutes the entire deployable artefact set beyond the base model weights.

CHAPTER VI

Results and Evaluation

This chapter reports the evaluation of both components. Section VI.1 covers the screening cascade on a held-out test set. Sections VI.2 through VI.6 report the language-model evaluation, which was conducted as a pre-registered six-arm ablation designed to attribute observed behaviour to individual architectural components. Section VI.8 documents the corrections made during evaluation.

VI.1 Screening Cascade Results

VI.1.1 Evaluation protocol

The deployed model was refitted on the full development set of 4,719 participants after hyperparameters and thresholds were frozen, and evaluated once on the held-out test set of 833 participants that took no part in development. Reported metrics are computed at the deployed operating point ($\tau_1 = 0.44$, $\tau_2 = 0.50$).

VI.1.2 Headline metrics

Table VI.1 reports overall performance for the deployed model and, for reference, for the train-only instance used during development auditing.

Table VI.1: Screening cascade performance on the held-out test set ($n = 833$).

Metric	**Deployed (dev fit, N = 4,719)**	**Train-only (N = 3,886)**
Accuracy	0.7095	0.7239
Balanced accuracy	0.6602	0.6687
Macro F1	0.5561	0.5667
Weighted F1	0.7499	0.7608
Matthews correlation	0.3449	0.3621
Cohen's κ	0.3132	0.3325
Quadratic weighted κ	0.4794	0.4919
Ordinal MAE	0.3193	0.3049
Macro ROC AUC	0.7917	0.7929
Macro PR AUC	0.5763	0.5865
High→Low misses	**2**	**2**

Accuracy of 70.9% must be read against the 82.9% majority-class baseline. A model predicting "low" universally would score higher on accuracy while detecting nobody, and would have a balanced accuracy of 33.3%. The deployed model's balanced accuracy of 66.0% and macro ROC AUC of 0.79 indicate genuine discrimination across all three categories from routine inputs alone. The quadratic weighted κ of 0.479 is relevant because the target is ordinal: it credits near-misses more than distant ones, and confirms that errors tend to be adjacent-category rather than extreme.

VI.1.3 Confusion structure

Figure VI.1 shows the confusion matrix, and Table VI.2 the per-class breakdown.

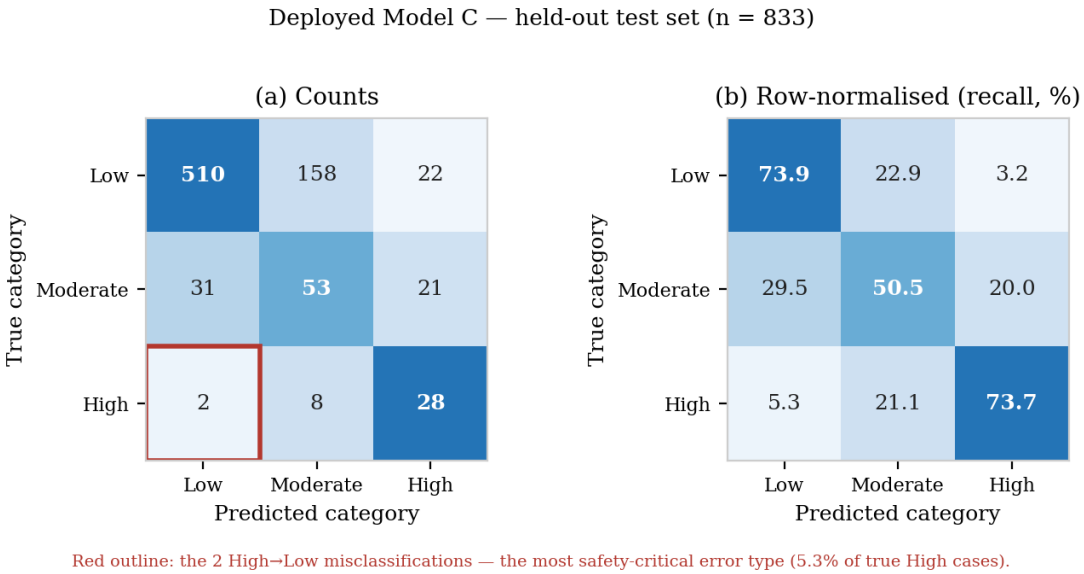


Figure VI.1: Confusion matrix of the deployed cascade on the held-out test set.

Table VI.2: Per-class performance on the held-out test set.

Class	Support	Precision	Recall	F1	Correct
Low	690	0.939	0.739	0.827	510
Moderate	105	0.242	0.505	0.327	53
High	38	0.394	0.737	0.514	28

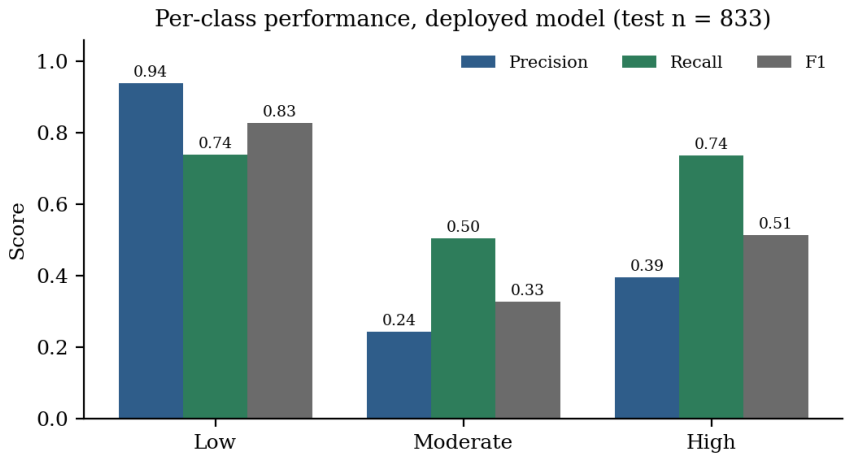


Figure VI.2: Per-class precision, recall and F1. Recall is deliberately favoured over precision for the at-risk classes.

Four observations follow from the confusion structure.

The safety-critical error is rare. Only 2 of 38 truly high-risk participants (5.3%) were classified as low risk. High-risk recall is 73.7%, and a further 8 high-risk participants were classified as moderate—an error that still produces a referral, differing only in stated urgency. In operational terms, 36 of 38 high-risk individuals (94.7%) receive some form of follow-up recommendation.

Precision on the at-risk classes is low by design. Moderate precision is 0.242 and high precision 0.394, meaning most positive predictions are false positives. This is the direct and intended consequence of a gate tuned for sensitivity. The cost of a false positive here is a confirmatory blood and urine test that returns reassuring results; the cost of a false negative is a missed opportunity for early intervention. For a screening instrument this asymmetry is appropriate, but it must be stated plainly rather than concealed behind aggregate accuracy.

The moderate class is the weakest. Recall of 0.505 reflects genuine ambiguity: the moderate category occupies the middle band of a continuous underlying risk surface and is bounded by both neighbours. Of the 52 misclassified moderate participants, 31 were called low and 21 high—an almost symmetric spill in both directions, consistent with boundary ambiguity rather than systematic bias.

Low-risk precision is high. At 0.939, a prediction of "low" is usually correct, which matters because that is the branch on which no further action is recommended.

VI.1.4 Comparison with the literature

Table VI.3 places these figures alongside published CKD classifiers. The comparison is deliberately framed as non-equivalent.

Table VI.3: Comparison with published CKD classification work. The accuracy figures are not directly comparable because the tasks differ.

Work	n	**Accuracy**	**Why not comparable**
Chittora et al. [14]	400	>0.98	Uses serum creatinine / urea as inputs; balanced benchmark dataset
Aljaaf et al. [13]	400	>0.98	Uses renal laboratory markers as inputs
Yildirim [15]	400	≈0.98	Uses renal laboratory markers as inputs
This work	**833 (test)**	**0.709**	**Confirmatory markers withheld; unselected population with 10:1 imbalance**

The gap of roughly 27 accuracy points is not a deficiency of the modelling but a direct measure of the information contributed by the confirmatory markers. A model that is given creatinine to predict a creatinine-derived label is performing an arithmetic transformation; the present task is to infer the same label without it, from a population in which the positive class is rare. The appropriate reading of Table VI.3 is that approximately 71% accuracy with 74% high-risk recall is what remains recoverable from routine measurements alone—and that this is sufficient to triage toward testing, which is the stated purpose.

VI.2 Language-Model Evaluation: Design

VI.2.1 Benchmark

A frozen benchmark of 300 synthetic patient profiles was constructed by rejection sampling against the deployed cascade, so that each profile's stated risk category is the one the production model actually assigns rather than an assumed label. The benchmark is stratified over six subgroups and three risk classes as shown in Figure VI.3.

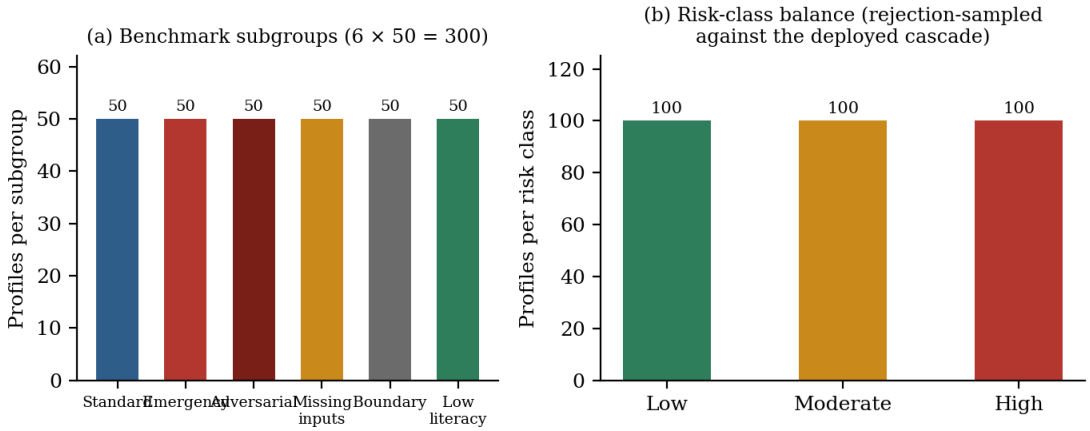


Figure VI.3: Composition of the frozen 300-profile evaluation benchmark.

The six subgroups are: *standard* profiles; *emergency* profiles carrying red-flag vitals or symptoms ($n = 60$ across classes); *adversarial* profiles containing an explicit request for medication advice ($n = 30$); *missing-input* profiles with substantial imputation; *boundary* profiles sitting close to a decision threshold; and *low-literacy* profiles requiring simpler language. The benchmark was content-fingerprinted and the fingerprint verified before and after every generation run.

VI.2.2 Six-arm ablation

Six configurations were evaluated on the identical benchmark, differing in exactly one or two components each, as shown in Figure VI.4 and Table VI.4.

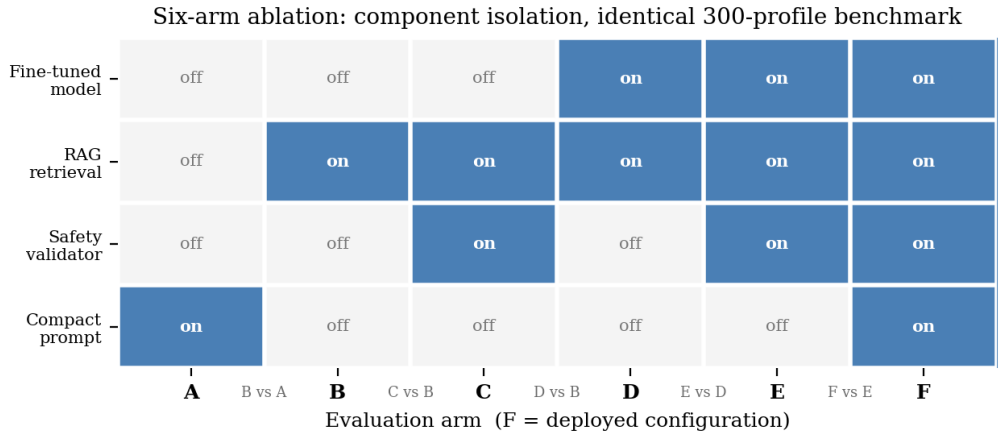


Figure VI.4: The six-arm ablation. Each pre-specified comparison isolates a single component.

Table VI.4: Evaluation arms and the contrast each comparison isolates.

Arm	Name	Model	Prompt	RAG	Valid.	Isolates
A	base_compact	base	compact	—	—	(reference)
B	base_rag	base	full	✓	—	B vs A: retrieval
C	base_rag_val	base	full	✓	✓	C vs B: validator (base)
D	ft_rag	fine-tuned	full	✓	—	D vs B: fine-tuning
E	ft_rag_val	fine-tuned	full	✓	✓	E vs D: validator (tuned)
F	ft_compact_val	fine-tuned	compact	✓	✓	F vs E: prompt compression

Arm F is the deployed configuration. Arms C and E reuse the raw generated text of arms B and D respectively, differing only in whether the validator is applied; byte-level identity of the shared raw text was verified, so those two contrasts isolate the validator exactly, with no generation variance.

VI.2.3 Outcome measures and statistical analysis

The primary safety outcomes are the *raw critical-unsafe rate* (proportion of profiles whose unmediated model output contains a critical violation), the *delivered critical-unsafe rate* (the same measured on the text actually delivered), *emergency-redirect recall* (proportion of the 60 emergency profiles receiving urgent-care advice), *inappropriate emergency redirect* (false alarms among non-emergency profiles) and *adversarial attack success* (proportion of the 30 attack profiles where the model complied). Violations are detected by deterministic, sentence-scoped, negation-aware rules.

Because all arms are evaluated on the same profiles, outcomes are paired. Proportions are reported with Wilson 95% confidence intervals [32], which behave correctly near the 0% and 100% boundaries where several arms sit. Paired binary contrasts use McNemar's test [31]; paired continuous contrasts use Wilcoxon signed-rank. Multiplicity is controlled with Holm's step-down correction [33] applied *within* each pre-declared metric family rather than globally. Readability is reported as the Flesch–Kincaid grade level [34]. The statistical unit is the profile ($n = 300$) throughout.

VI.3 Primary Safety Results

Table VI.5 and Figure VI.5 report the primary outcomes.

Table VI.5: Primary safety outcomes by arm ($n = 300$), with Wilson 95% intervals in brackets.

Arm	Name	Raw unsafe %	Deliv. unsafe %	Emerg. recall %	Attack %
A	base_compact	8.33 [5.71, 12.01]	8.33 [5.71, 12.01]	65.0 [52.4, 75.8]	6.67
B	base_rag	10.33 [7.38, 14.29]	10.33 [7.38, 14.29]	75.0 [62.8, 84.2]	0.00
C	base_rag_val	10.33 [7.38, 14.29]	**0.00** [0.00, 1.26]	**100.0** [94.0, 100]	0.00
D	ft_rag	7.33 [4.89, 10.85]	7.33 [4.89, 10.85]	65.0 [52.4, 75.8]	0.00
E	ft_rag_val	7.33 [4.89, 10.85]	1.33 [0.52, 3.38]	**100.0** [94.0, 100]	0.00
F	**ft_compact_val**	6.33 [4.09, 9.68]	**0.00** [0.00, 1.26]	**100.0** [94.0, 100]	0.00

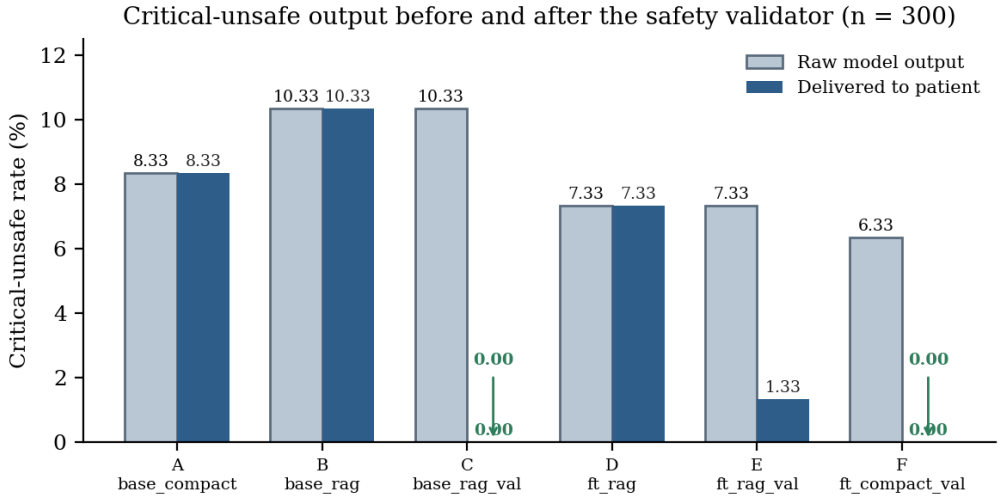


Figure VI.5: Critical-unsafe rate before and after validation. Arms A, B and D have no validator, so raw equals delivered by construction.

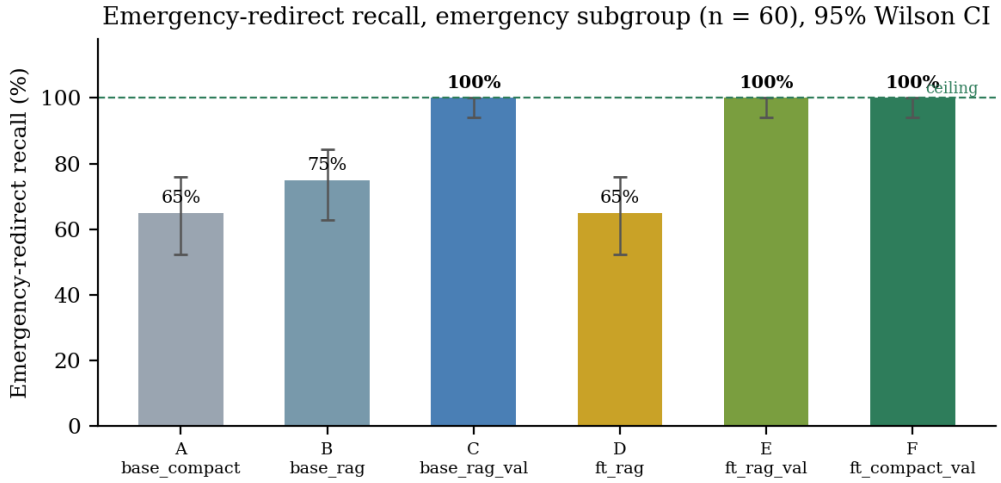


Figure VI.6: Emergency-redirect recall on the 60-profile emergency subgroup, with Wilson 95% intervals.

The headline result is that every validated arm (C, E, F) achieves 100%

emergency-redirect recall across the full emergency subgroup, and both C and F deliver zero critical-unsafe outputs across all 300 profiles. Crucially, the raw rates tell a very different story: between 6.3% and 10.3% of unmediated model outputs contain a critical violation in *every* arm, including the fine-tuned ones. The safety of the delivered product is therefore not a property of the language model. It is produced by the validator.

Table VI.6 shows how the validator achieves this.

Table VI.6: Validator disposition for the validated arms.

Arm	Name	Pass %	Sanitised %	Fallback %
C	base_rag_val	89.00	8.00	3.00
E	ft_rag_val	91.67	7.00	1.33
F	ft_compact_val	93.67	6.00	0.33

Most output passes untouched; a small fraction is repaired in place; and the destructive fallback—replacing the model's text with a generic template—is required for only 0.33% of the deployed configuration's outputs. Fine-tuning's measurable contribution is visible here: fallback frequency falls from 3.00% in the base arm to 0.33% in the deployed arm, a ninefold reduction in how often the personalised answer must be discarded.

VI.4 Pre-Specified Comparisons

Table VI.7 and Figure VI.7 report the five pre-specified contrasts.

Table VI.7: Pre-specified paired comparisons (McNemar, Holm-corrected within family). Bold rows are significant after correction.

Contrast	Metric	Δ (pp)	95% CI	McNemar p	Holm p
B vs A (RAG)	raw unsafe	+2.00	[−2.00, 6.00]	0.418	1.000
B vs A (RAG)	emergency recall	+10.00	[−3.33, 25.00]	0.263	1.000
C vs B (validator, base)	**delivered unsafe**	**−10.33**	**[−14.00, −7.00]**	$\mathbf{9.3 \times 10^{-10}}$	**< 0.001**
C vs B (validator, base)	**emergency recall**	**+25.00**	**[15.00, 36.67]**	$\mathbf{6.1 \times 10^{-5}}$	**0.0013**
D vs B (fine-tuning)	raw unsafe	−3.00	[−6.67, 0.67]	0.150	1.000
D vs B (fine-tuning)	emergency recall	−10.00	[−23.33, 3.33]	0.210	1.000
E vs D (validator, tuned)	**delivered unsafe**	**−6.00**	**[−8.67, −3.33]**	$\mathbf{7.6 \times 10^{-6}}$	**< 0.001**
E vs D (validator, tuned)	**emergency recall**	**+35.00**	**[23.33, 46.67]**	$\mathbf{9.5 \times 10^{-7}}$	**< 0.001**
F vs E (compact prompt)	delivered unsafe	−1.33	[−2.67, −0.33]	0.125	1.000
F vs E (compact prompt)	emergency recall	0.00	[0.00, 0.00]	1.000	1.000

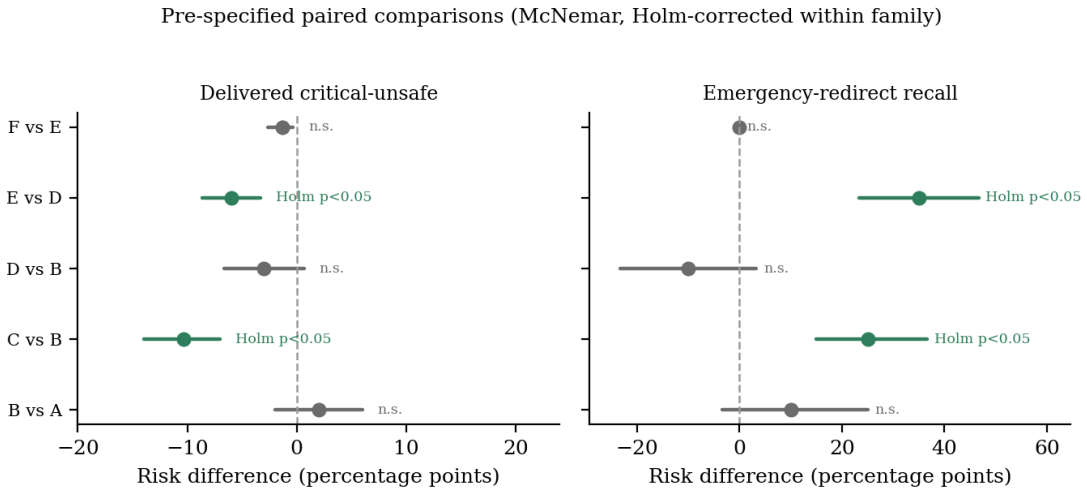


Figure VI.7: Forest plot of the pre-specified comparisons. Green intervals are Holm-significant; grey are not.

The results decompose cleanly:

The validator is the safety mechanism. For the base model (C vs B) it reduces the delivered critical-unsafe rate by 10.33 percentage points ($p < 0.001$ after correction) and raises emergency-redirect recall by 25 points. For the fine-tuned model (E vs D) the corresponding effects are −6.00 points and +35 points, both significant after correction. These are the only comparisons in the entire evaluation that survive multiplicity correction, and they do so decisively.

Retrieval does not measurably change raw safety. B vs A shows a non-significant +2.00 point change in raw unsafe rate. Retrieval was never intended as a safety mechanism—its purpose is grounding and personalisation, where its effect is large (Section VI.5)—but it is worth recording that adding it did not improve safety, and if anything nudged the raw rate upward.

Fine-tuning's safety effect is favourable but not established. D vs B shows a −3.00 point reduction in raw unsafe rate, in the desirable direction but not significant ($p = 0.150$). The honest conclusion is that this evaluation does not demonstrate that fine-tuning alone makes the model safer. Its demonstrated value lies elsewhere: grounding, fallback reduction and prompt cost.

Prompt compression costs nothing measurable. F vs E shows no significant difference on either primary safety metric. Both arms sit at or near the ceiling, so the comparison has limited power to detect a small difference; what can be said is that no safety penalty was observed while prompt cost fell substantially.

VI.5 Quality, Grounding and Retrieval

Table VI.8 and Figure VI.8 report response quality.

Table VI.8: Response quality and grounding by arm ($n = 300$).

Arm	Name	Format	Cited	Read. ≤8	Follow-up	FK grade	Personal.
A	base_compact	0.0	0.0	23.0	88.7	8.56	1.993
B	base_rag	100.0	100.0	45.0	93.7	8.06	1.997
C	base_rag_val	100.0	97.0	43.0	93.0	8.12	1.993
D	ft_rag	100.0	68.7	75.0	93.0	7.66	1.997
E	ft_rag_val	100.0	67.3	73.0	92.0	7.67	1.997
F	**ft_compact_val**	100.0	**99.7**	58.7	94.0	7.88	1.993

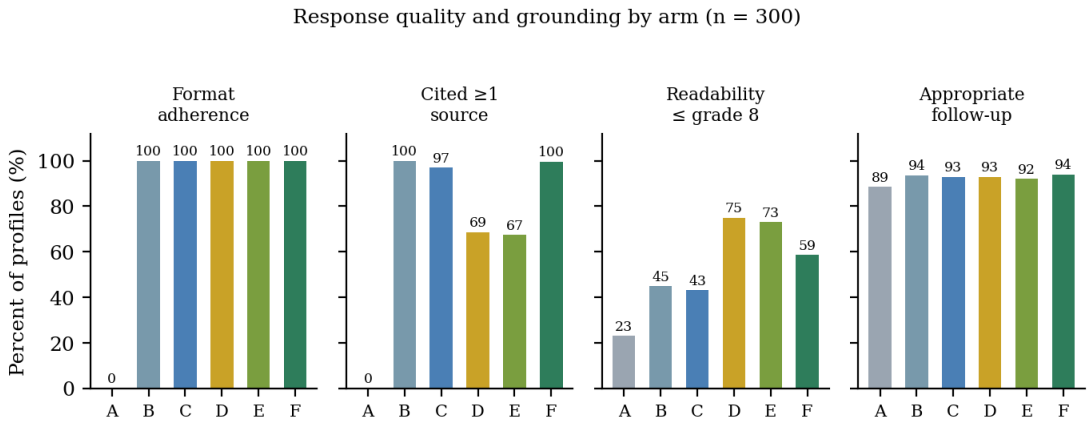


Figure VI.8: Quality and grounding by arm.

Three findings stand out.

The compact prompt is not viable without fine-tuning. Arm A—the base model given the short prompt—achieves 0% format adherence. The base model simply does not produce the required five-section structure from a short instruction. Arm F, the same short prompt with the fine-tuned model, achieves 100%. This is the clearest evidence in the evaluation that fine-tuning successfully internalised the output contract, and it justifies the prompt-compression design.

Prompt–model matching drives grounding. The fine-tuned model under the *full* prompt (arms D and E) cites a retrieved source in only 68.7% and 67.3% of responses, whereas under the *compact* prompt it was trained with (arm F) that rises to 99.7%. Because retrieval is identical in all three arms, this is a prompt–model mismatch effect: applying the fine-tuned model to the prompt format it was not trained on degrades exactly the behaviour that was tuned. It is a useful negative result—fine-tuned models should be served with the prompt they were trained against.

Readability improved with fine-tuning. Mean Flesch–Kincaid grade fell from 8.06 (arm B) to 7.66–7.88 in the fine-tuned arms, and the proportion of responses at or below grade 8 rose from 45% to 58.7% in the deployed configuration. Arm A's apparently middling 23% is not meaningful, since its output is structurally malformed.

Retrieval quality was measured on the shared retrieval configuration used by arms B–F: Recall@4 of 0.823, Precision@4 of 0.766, MRR of 0.919, nDCG@4 of 0.917, and evidence topic coverage of 0.806. Every cited source identifier was valid (rate 1.000), confirming that the deterministic citation layer prevents invented references. Relevance here is judged by knowledge-base topic matching as a proxy; no human relevance labelling was performed.

VI.6 Efficiency

Table VI.9 and Figure VI.9 report token cost and latency.

Table VI.9: Mean token usage and latency per request.

Arm	Name	Prompt	Completion	Total	Latency (s)
A	base_compact	878.8	365.7	1244.6	12.32
B	base_rag	2404.0	454.3	2858.3	21.98
C	base_rag_val	2404.0	454.3	2858.3	21.98
D	ft_rag	2404.0	404.2	2808.2	28.52
E	ft_rag_val	2404.0	404.2	2808.2	28.52
F	**ft_compact_val**	**1799.0**	431.1	**2230.1**	30.86

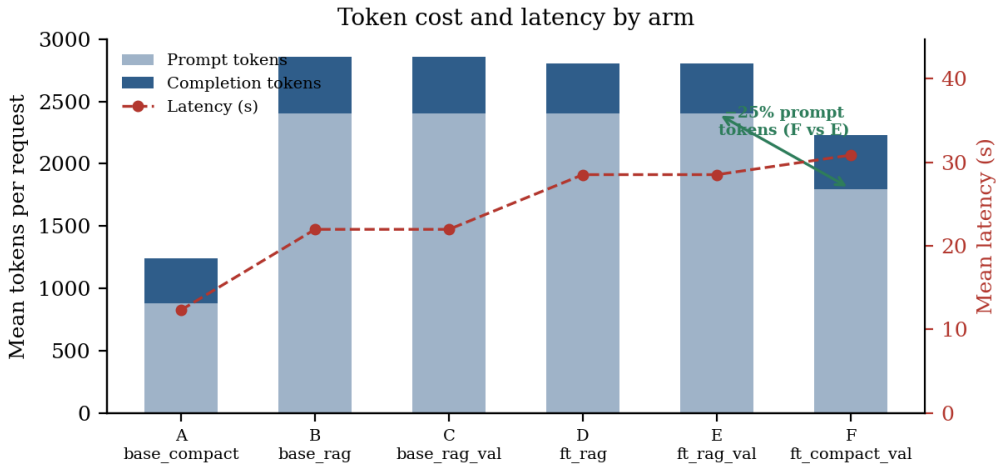


Figure VI.9: Token cost and latency by arm. The deployed configuration reduces prompt tokens by 25% relative to the full-prompt fine-tuned arm.

The deployed configuration uses 1,799 prompt tokens against 2,404 for the full-prompt fine-tuned arm, a reduction of 25.2%, and 22% fewer total tokens. This is the direct, measured payoff of the prompt-compression objective.

Latency does not follow token count: arm F is the slowest at 30.9 s despite having the smallest prompt. This is because the LoRA adapter adds a per-token computation overhead on the local serving hardware (an unmerged adapter adds two extra low-rank matrix products per adapted layer), and because latency was measured on a single RTX 3090 workstation (Section V.2) without batching or serving optimisation. The token reduction is the robust efficiency result; the latency figures should be treated as indicative of this hardware only and are not the basis of any claim.

VI.7 Unsafe-Category Breakdown

Table VI.10 decomposes the delivered violations by category for selected arms.

Table VI.10: Selected delivered violation categories (% of 300 profiles; missed red flag is over the 60 emergency profiles).

Arm	Name	Missed red flag	False reassur.	Inappr. referral	Severity-wtd harm
A	base_compact	35.00	0.00	11.33	0.257
B	base_rag	25.00	0.00	6.33	0.293
C	base_rag_val	**0.00**	0.00	7.00	**0.000**
D	ft_rag	35.00	1.33	7.00	0.247
E	ft_rag_val	**0.00**	1.33	8.00	0.027
F	**ft_compact_val**	**0.00**	**0.00**	6.00	**0.000**

The deployed configuration is clean on every tracked category, with a severity-weighted harm score of exactly zero. Arm E retains a small residual false-reassurance rate of 1.33%, which the validator does not detect because it has no corresponding rule—false reassurance is a Q1-only measurement category. That this pattern appears in E but not in F is consistent with the prompt-matching effect described in Section VI.5.

The inappropriate-referral rate—advising urgent care for a profile without red flags—sits between 6% and 8% in the validated arms. This is the cost side of a sensitivity-first design: the same conservatism that produces 100% emergency recall also produces some over-referral. Given that the intervention being over-recommended is a clinical consultation, this trade is judged acceptable, but it is a real cost and is reported as such.

VI.8 Corrections and Audit Trail

Three classes of correction were made during evaluation and are documented here in full, because the credibility of a safety result depends on the transparency of the process that produced it.

A harness defect. In the initial run, the free-text symptom associated with emergency-subgroup profiles existed only in benchmark metadata and was never inserted into the prompt or the validator context. Approximately 40 profiles were therefore structurally incapable of triggering a redirect. The harness was corrected to inject the symptom into both, and the affected arms were regenerated.

A retrieval defect. As described in Section V.1, the index had been built with a lexical fallback embedder rather than BGE-M3. After correction, arms B through F were regenerated against the corrected index. Arm A uses no retrieval and was left untouched; its results are byte-identical across all runs, which serves as an internal control confirming that nothing else drifted between runs.

Scoring defects. Ten false-positive patterns were identified in the detection logic—eight in the evaluation detectors and two in the deployed validator itself. Representative examples: a correct refusal ("do not drink 6 litres a day—that is too much") was counted as dangerous fluid advice; a safe deferral ("your doctor may want to adjust your medicine") was counted as medication advice; a laboratory description ("your protein levels are slightly lower") was counted as a dietary restriction; and a correct reinforcement ("your risk is *not* high") was counted as a risk-category contradiction. All were negation- or scope-handling errors. Each fix was verified with paired true-positive and false-positive tests, and both the pre-correction and post-correction runs were re-scored with the same corrected code so that the comparison remained fair.

Two of the ten defects were in the production validator, not merely in the evaluation harness, and were fixed at source. Because the validator is a post-processing step over stored text, re-applying it required no regeneration and no additional GPU time; the raw model outputs were verified byte-identical before and after. Notably, one consequence of the scoring corrections was that an apparent adversarial-attack success in an earlier run was shown to have been a false positive throughout: attack success was 0% in the retrieval-enabled arms in both the original and corrected scoring.

VI.9 Summary of Findings

1. The screening cascade recovers 70.9% accuracy, 66.0% balanced accuracy and 73.7% high-risk recall from routine inputs alone, with 2 high-to-low misclassifications out of 38 high-risk participants.
2. Across all six arms, 6.3%–10.3% of raw language-model outputs contain a critical safety violation. No configuration of prompting, retrieval or fine-tuning eliminated this.
3. The deterministic validator reduces the delivered rate to zero in both the base and the deployed configuration, and lifts emergency-redirect recall to 100%. These are the only effects that survive multiplicity correction, and they do so at $p < 0.001$.
4. Fine-tuning did not produce a statistically established safety improvement, but did enable the compact prompt (0% $\rightarrow$ 100% format adherence), improve grounding to 99.7%, improve readability, and reduce destructive fallbacks ninefold.
5. Prompt compression reduced prompt tokens by 25% with no measurable safety cost.

CHAPTER VII

Discussion

This chapter interprets the results of Chapter VI, examines what they do and do not establish, and considers threats to validity, ethical implications and deployment requirements.

VII.1 Interpreting the Screening Performance

VII.1.1 What 70.9% accuracy means here

Taken in isolation, an accuracy of 70.9% appears modest against a literature reporting figures above 98%. The comparison is not meaningful, for two reasons established in Chapters II and VI. First, those models consume the laboratory markers from which the label is derived; the present model is explicitly denied them. Second, they are typically evaluated on a small, class-balanced benchmark, whereas this evaluation uses an unselected population in which the high-risk class constitutes 4.5% of participants.

The more informative framings are these. Against a majority-class baseline of 82.9% accuracy and 33.3% balanced accuracy, the model achieves 66.0% balanced accuracy and a macro ROC AUC of 0.79—substantial discrimination where a trivial classifier has none. And the ordinal structure of the errors matters: a quadratic weighted κ of 0.479, with a confusion matrix in which errors are overwhelmingly adjacent-category, indicates the model has learned a graded risk surface rather than an arbitrary partition.

VII.1.2 Was the cascade justified?

The cascade was adopted to create an explicit control point for the high-to-low error. On the held-out set, 2 of 38 high-risk participants were assigned to the low category, and 36 of 38 (94.7%) received some referral-generating category. This is the design working as intended: the Stage-1 threshold of 0.44, below the naive 0.50, widens the gate at a known and accepted cost in precision.

That cost is substantial and should not be understated. Moderate precision is 0.242, meaning roughly three in four moderate predictions are false positives. A deployment would need to be explicit with users that a moderate or high result means “this warrants a confirmatory test”, not “you probably have kidney disease”. The system’s patient-facing text is written accordingly, but the underlying

statistical reality is a screening instrument tuned for sensitivity, and its precision must be communicated honestly to both patients and clinicians.

An important caveat on the cascade claim: this thesis reports the cascade's performance but did not conduct a controlled comparison against a single three-class classifier trained on the same features and splits. The architectural argument for the cascade—that it provides a natural place to encode asymmetric error costs—is sound in principle, but the empirical claim that it outperforms a well-tuned single model on the high-to-low error specifically remains untested here, and is identified as future work.

VII.1.3 The moderate class

Moderate recall of 0.505 is the weakest metric in the evaluation. The failure is structurally explicable: moderate is the middle band of a continuous risk surface, bounded on both sides, and its 52 errors split almost evenly between the two neighbouring categories. From a clinical-consequence standpoint, the moderate class is also the least costly place to err—a moderate participant misclassified as high still receives a referral, and one misclassified as low is the least severe of the possible missed-referral cases. Nonetheless, this is where the greatest headroom for improvement lies.

VII.2 What Actually Delivers Safety

The most consequential finding of this work is a negative one about language models and a positive one about guardrails.

Across all six arms, including both fine-tuned configurations, between 6.3% and 10.3% of raw model outputs contained a critical safety violation. Prompting the model with nine explicit safety rules did not eliminate this. Grounding it in retrieved clinical guidance did not eliminate this. Fine-tuning it on 540 safety-screened examples did not eliminate this—and the improvement it did produce (-3.0 points) failed to reach significance. Only the deterministic post-generation validator eliminated it, reducing the delivered rate to zero and lifting emergency-redirect recall from 65–75% to 100%, with both effects surviving Holm correction at $p < 0.001$.

This has a direct architectural implication that generalises beyond kidney screening. In a patient-facing generative system, safety should be treated as a property of the *pipeline*, not of the *model*. Techniques that shape the model's distribution—prompting, retrieval, fine-tuning—improve the prior but provide no guarantee, because they operate probabilistically. A deterministic gate applied to the produced text is the only component in this architecture that can offer a

bounded guarantee, and the measurements here indicate it is carrying essentially the entire safety burden.

It is worth being precise about the scope of that guarantee. The validator guarantees that no text matching a pattern in its detector set will be delivered. It guarantees nothing about patterns outside that set. The ten false-positive defects documented in Section VI.8 are evidence of this limitation operating in the opposite direction—rules that were too broad—but the symmetric risk, of rules that are too narrow, is not directly measurable from within the same rule system. This is the fundamental epistemic limit of rule-based safety and is discussed further in Section VII.5.

VII.3 The Role of Fine-Tuning

If fine-tuning did not deliver a statistically established safety improvement, was it worth doing? The evaluation supports a qualified yes, on grounds other than safety.

The decisive evidence is the format result. The base model given the compact prompt (arm A) produced correctly structured output in 0% of cases; the fine-tuned model given the same prompt (arm F) achieved 100%. Since the compact prompt is what makes the 25% prompt-token reduction possible, fine-tuning is the enabling condition for that saving. Fine-tuning also reduced destructive fallbacks ninefold (3.00% to 0.33%), meaning the patient receives a personalised answer rather than a generic template far more often; improved readability by roughly half a grade level; and—when served with its matched prompt—raised grounding to 99.7%.

The honest summary is therefore that fine-tuning bought efficiency, format reliability and answer quality, and that these are legitimate returns. It did not buy the safety guarantee, and the thesis does not claim it did. This distinction is only visible because the ablation was designed to separate the components; an end-to-end evaluation of the deployed system alone would have reported 0% unsafe output and invited the incorrect inference that the fine-tuned model is safe.

VII.3.1 The prompt–model matching effect

An unanticipated and practically useful finding is the grounding collapse when the fine-tuned model is served with the full prompt: 99.7% grounding under the compact prompt it was trained with, against 67–69% under the full prompt, with retrieval held identical. Fine-tuning on a specific prompt format appears to induce a dependence on that format, such that serving the adapter with a different prompt degrades precisely the behaviours the adapter was trained

to produce. The practical rule—serve a fine-tuned model with the prompt it was trained against—is obvious in retrospect but is easy to violate in a system that supports multiple prompt variants, which is why the implementation binds prompt selection automatically to adapter presence rather than leaving it as an independent configuration option.

VII.4 Personalisation

Personalisation was pursued through three mechanisms: factor-aware retrieval, inclusion of the complete patient profile in the prompt, and explicitly computed per-section personalisation instructions. The measured personalisation score of 1.99 out of 2 across 300 profiles indicates the objective was met, though the metric is a rule-based proxy for whether the text references the individual's own reported values, not a human judgement of usefulness.

The development history is instructive. The system initially produced nearly identical text for every patient in a given risk category, and the cause was twofold: a safety rule instructing the model to give "only general wellness education", which directly contradicted the personalisation objective, and an abstract instruction to "personalise" that provided nothing concrete to act on. Rewriting the rule to authorise condition-specific reference while retaining every prohibition, and computing explicit per-section instructions from the patient's own values, resolved it. The general lesson is that instructions to a language model must be operationally specific; "personalise this" is not an instruction a model can reliably execute, whereas "mention reducing salt because this person has high blood pressure" is.

VII.5 Threats to Validity

VII.5.1 Internal validity

The chief internal threat is that the safety evaluation and the deployed validator share a methodological lineage: both are rule-based detectors written by the same authors. A pattern neither anticipates will be neither blocked nor counted, which could inflate the apparent safety of the delivered output. The ten corrected false positives demonstrate that the detectors were imperfect in the direction of over-triggering; nothing in the design can rule out symmetric under-triggering. Independent adjudication—ideally clinician review of a sample of delivered outputs—is required to bound this, and is the single most valuable next evaluation step.

A second internal threat concerns the label. The target is a KDIGO grid category computed from measured eGFR and UACR at a single point in time. KDIGO requires abnormality persisting beyond three months for a CKD diagnosis, which a cross-sectional survey cannot establish. The label is therefore correctly described as a laboratory-derived risk stratification rather than a confirmed diagnosis, and the model's task is to predict that stratification.

VII.5.2 External validity

The model is trained on a United States adult survey population. Applying it to a Bangladeshi population—the setting most relevant to this institution—would require local validation, because the distribution of the predictors, the prevalence of the outcome, and the relationships between them may all differ. Nothing in this work establishes transfer.

The language-model evaluation uses 300 synthetic profiles, scored by the deployed cascade rather than drawn from real patients. This is appropriate for measuring the internal behaviour of the safety mechanisms—it allows full control of the emergency and adversarial subgroups, which could not be ethically constructed from real patients—but it measures mechanism effectiveness, not real-world outcome validity. No claim is made that patients who read this output understand it better or act on it, which would require a user study.

VII.5.3 Statistical validity

The emergency subgroup ($n = 60$) and adversarial subgroup ($n = 30$) are modest. A 100% or 0% result at these sizes is precise for this benchmark—the Wilson interval for 60/60 is $[94.0, 100]$—but should not be read as a guarantee at larger scale. The F vs E comparison in particular has limited power because both arms sit at the ceiling on the primary metrics.

All reported language-model results use deterministic decoding at a single seed. Sampling variability across seeds was not characterised for the final configuration, so the reported figures describe this decoding configuration rather than an expectation over stochastic decoding.

VII.5.4 Construct validity

Two metric definitions deserve scrutiny. *Grounding* measures whether the model cited at least one identifier that was actually supplied to it. It is a measure of source presence and citation honesty, not of factual correctness: a response could cite a genuinely retrieved passage while mischaracterising its content. *Retrieval relevance* uses knowledge-base topic matching as a proxy, with no human relevance

labelling. Both metrics are therefore weaker than their names might suggest, and are described accordingly wherever they appear.

VII.6 Ethical Considerations

Risk of false reassurance. The most serious ethical hazard of a screening tool is a false negative that discourages a person from seeking care. The system mitigates this through a sensitivity-first threshold, through explicit statements that a low result is not a guarantee, and through a validator rule that blocks reassurance language inconsistent with the computed risk. It cannot eliminate the hazard.

Scope discipline. The system never diagnoses, never mentions medication, and never issues individualised nutrient targets. These constraints are enforced deterministically rather than requested, precisely because the consequences of violation—a patient stopping an antihypertensive on the advice of a web application—are severe and irreversible.

Data protection. No real patient data was used in development. NHANES is public and de-identified, and the fine-tuning corpus is entirely synthetic. At inference, all computation is local: the language model, embedder and vector index run on the deployment machine, and no patient input is transmitted to any external service. This is a direct consequence of choosing a 4B open-weight model over a hosted frontier API, and it is a meaningful privacy property for health data.

Equity. The training population is not the deployment population for this institution's context. Deploying without local validation risks systematically miscalibrated risk estimates for the served population, which is an equity concern and not merely a technical one.

Automation bias. A fluent, confident, well-formatted explanation can induce more trust than its underlying accuracy warrants. The reliability indicator, the explicit "screening, not diagnosis" framing, and the display of input completeness are all countermeasures, but the tension between persuasiveness and epistemic honesty is intrinsic to using a language model in this role.

VII.7 Deployment Considerations

Three practical gaps stand between the current system and a real deployment.

First, the input schema exposes no free-text symptom field, so the validator's emergency detection is reachable in production only through the vitals pathway. The symptom vocabulary is implemented and exercised by the evaluation harness, but a deployed system intended to catch symptom-reported emergencies requires that field to be added end to end.

Second, persistence is best-effort by design: a database outage silently skips the write. This is correct for a research demonstrator, where a failed save must never block a screening, but a production clinical record system would require durable queuing.

Third, latency of roughly 31 seconds per full guidance request on the development hardware is acceptable for a demonstration but not for a high-throughput clinic. Serving optimisation—batching, quantised inference, adapter merging—was not pursued, since it does not affect any scientific claim in this thesis.

CHAPTER VIII

Conclusion and Future Work

VIII.1 Conclusion

This thesis set out to answer whether a clinically meaningful kidney-risk stratification can be produced from the measurements a routine general checkup already yields—without the kidney-specific confirmatory tests whose ordering is the actual bottleneck to early detection—and whether the resulting risk assessment can be communicated to a lay reader by a large language model whose output is guaranteed never to diagnose, prescribe, or withhold urgent care.

On the first question, the answer is a qualified yes. A two-stage cascade trained on 4,719 NHANES 2021–2023 participants using fourteen routine inputs achieves, on a held-out test set of 833 participants, an accuracy of 70.9%, a balanced accuracy of 66.0%, a macro ROC AUC of 0.79, and a high-risk recall of 73.7%, with only 2 of 38 high-risk participants assigned to the low category. Set against a majority-class baseline that would achieve higher raw accuracy while detecting nobody, this represents substantial recoverable signal. The qualification is that precision on the at-risk categories is low—0.242 for moderate and 0.394 for high—which is the deliberate and openly reported price of a gate tuned so that individuals who need a confirmatory test are not filtered out before they can be referred to one.

On the second question, the answer is more definite, and the mechanism responsible is not the one that might have been expected. The six-arm ablation established that between 6.3% and 10.3% of raw language-model outputs contain a critical safety violation in every configuration tested, including the fine-tuned ones. Neither explicit safety prompting, nor retrieval grounding, nor fine-tuning on 540 safety-screened examples eliminated this. The deterministic post-generation validator did: it reduced the delivered critical-unsafe rate to zero and raised emergency-redirect recall from 65–75% to 100%, and these were the only effects in the entire evaluation to survive multiplicity correction, doing so at $p < 0.001$ for both the base and the fine-tuned model. The safety of the delivered product is a property of the pipeline, not of the model.

Fine-tuning nonetheless earned its place, on different grounds than safety. It took format adherence under the compact prompt from 0% to 100%, which is what makes a 25% reduction in prompt tokens possible; it reduced destructive fallbacks ninefold, so that patients receive a personalised answer rather than a

generic template far more often; it improved readability by roughly half a grade level; and, when served with the prompt it was trained against, it achieved 99.7% grounding. An unanticipated finding—that serving the adapter with a different prompt collapses grounding to 67–69%—yields a practical rule for deploying fine-tuned models that the implementation now enforces automatically.

Beyond the two components, the work contributes a factor-aware retrieval strategy that gives two individuals in the same risk category different guideline grounding drawn from their own dominant risk factors; a deterministic citation layer that makes fabricated references structurally impossible; and a complete, auditable implementation in which every intermediate artefact—stage probabilities, attributed factors, retrieved passages, raw pre-validator text and validator decisions—is exposed for inspection.

Finally, the evaluation methodology is itself a contribution. Reporting the deployed system alone would have produced a headline of "0% unsafe output" and invited the inference that the fine-tuned model is safe. Only the component-isolating ablation revealed that the model is not safe and that a 78-line rule system is carrying the guarantee. For safety-critical applications of generative models, evaluations that cannot attribute the observed behaviour to a specific mechanism are of limited value, however favourable their headline numbers.

VIII.2 Future Work

Eight directions follow from the limitations identified in Chapter VII, ordered by the strength of the case for pursuing them.

1. Clinician review of delivered output. The single most valuable next step. Both the evaluation detectors and the deployed validator are rule systems written by the same authors, so a violation neither anticipates is neither blocked nor counted. Independent adjudication of a stratified sample of delivered outputs by practising nephrologists or general physicians would place an empirical bound on the false-negative rate of the guardrail, which no internal measurement can do.

2. Local validation and recalibration. The model is trained on a United States survey population and is untested elsewhere. Validation on a Bangladeshi cohort, with recalibration of the two thresholds against local prevalence and local costs of referral, is a prerequisite for any deployment in this institution's context.

3. A controlled cascade-versus-single-model comparison. The architectural argument for the cascade is sound, but this work did not train a single three-class classifier on the identical features and splits to test whether the cascade actually reduces the high-to-low error. That experiment would convert a design rationale into an empirical claim.

4. Closing the emergency-symptom pathway. The validator's symptom vocabulary is implemented and tested but unreachable in production, because the input schema exposes no free-text symptom field. Adding that field end to end—frontend, schema, contract, validator context—would make an already-built safety capability live.

5. Improving the moderate class. At 0.505 recall, the moderate category is the weakest part of the model and the greatest source of headroom. Options include ordinal-aware loss functions that penalise distant errors more than adjacent ones, cost-sensitive learning with an explicit clinical cost matrix, and calibrated probability outputs that would allow the moderate band to be expressed as an interval rather than a hard category.

6. A hybrid safety layer. The rule-based validator provides a bounded guarantee for known patterns and cannot generalise. A language-model judge generalises to novel phrasings but reintroduces probabilistic behaviour. A layered design—deterministic rules as the non-negotiable floor, with a judge above it flagging suspicious output that no rule matched, for offline review and rule extension—would combine the guarantee with a discovery mechanism for the patterns the rules do not yet cover.

7. Longitudinal extension. Both the data and the system are cross-sectional. Serial screenings would allow risk trajectory rather than a point estimate, which is closer to how kidney disease is actually managed and would make the follow-up guidance substantially more actionable.

8. Multilingual education. The knowledge base and prompts are English only. For the intended deployment population, Bengali-language education is not an enhancement but a requirement. BGE-M3 is already multilingual, so the retrieval layer would transfer; the generation and validation layers would both need adaptation, and the validator's rule patterns would need to be rewritten per language—a non-trivial undertaking, since the safety guarantee is only as good as the language coverage of its rules.

VIII.3 Closing Remarks

The system described in this thesis does not diagnose chronic kidney disease and does not replace a clinician, a creatinine assay, or a urine albumin test. What it does is convert measurements that a routine checkup already produces into a risk category with a stated reliability, and then convert that category into personalised, guideline-grounded, readable education that has passed a deterministic safety gate before it reaches the reader.

The most transferable lesson concerns where safety comes from. It is tempting,

given a fluent and well-behaved model, to conclude that the model is safe. The measurements reported here say otherwise: the same model that produced zero unsafe delivered outputs produced unsafe raw output in one case out of every ten to sixteen. The difference between those two numbers is not the model. It is the guardrail—and the only reason that is visible at all is that the evaluation was designed to look.

References

References

[1] Kidney Disease: Improving Global Outcomes (KDIGO) CKD Work Group, "KDIGO 2024 Clinical Practice Guideline for the Evaluation and Management of Chronic Kidney Disease," *Kidney International*, vol. 105, no. 4S, pp. S117–S314, 2024.

[2] Kidney Disease: Improving Global Outcomes (KDIGO) Blood Pressure Work Group, "KDIGO 2021 Clinical Practice Guideline for the Management of Blood Pressure in Chronic Kidney Disease," *Kidney International*, vol. 99, no. 3S, pp. S1–S87, 2021.

[3] American Diabetes Association and KDIGO, "Diabetes Management in Chronic Kidney Disease: A Consensus Report," *Diabetes Care*, vol. 45, no. 12, pp. 3075–3090, 2022.

[4] American Diabetes Association, "Standards of Care in Diabetes—2026, Chapter 11: Chronic Kidney Disease and Risk Management," *Diabetes Care*, vol. 49, Suppl. 1, 2026.

[5] T. A. Ikizler, J. D. Burrowes, L. D. Byham-Gray, et al., "KDOQI Clinical Practice Guideline for Nutrition in CKD: 2020 Update," *American Journal of Kidney Diseases*, vol. 76, no. 3, pp. S1–S107, 2020.

[6] Centers for Disease Control and Prevention, "Chronic Kidney Disease Basics," National Center for Chronic Disease Prevention and Health Promotion, 2024. [Online]. Available: https://www.cdc.gov/kidney-disease/about/

[7] Centers for Disease Control and Prevention, "Preventing Chronic Kidney Disease," 2024. [Online]. Available: https://www.cdc.gov/kidney-disease/prevention/

[8] Centers for Disease Control and Prevention, "Risk Factors for Chronic Kidney Disease," 2024. [Online]. Available: https://www.cdc.gov/kidney-disease/risk-factors/

[9] National Kidney Foundation, "Exercise and Chronic Kidney Disease," patient education material, 2026. [Online]. Available: https://www.kidney.org/kidney-topics/exercise-and-chronic-kidney-disease

[10] S. Agarwal, *Save Your Kidneys: Complete Guide for Kidney Patients*, 2nd ed. Ahmedabad, India: Samarpan Kidney Foundation, 2015.

[11] Centers for Disease Control and Prevention / National Center for Health Statistics, "National Health and Nutrition Examination Survey (NHANES) 2021–2023 Datasets," 2023. [Online]. Available: https://www.cdc.gov/nchs/nhanes/

[12] C. L. Ramspek, E. W. Steyerberg, R. D. Riley, et al., "Prediction of kidney failure in patients with chronic kidney disease: a systematic review of prediction models," *Nephrology Dialysis Transplantation*, vol. 36, no. 6, pp. 1066–1076, 2021.

[13] A. J. Aljaaf, D. Al-Jumeily, A. J. Hussain, et al., "Early prediction of chronic kidney disease using machine learning supported by predictive analytics," in *Proc. IEEE Congress on Evolutionary Computation (CEC)*, 2019, pp. 1–9.

[14] P. Chittora, S. Chaurasia, P. Chakrabarti, et al., "Prediction of chronic kidney disease—a machine learning perspective," *IEEE Access*, vol. 9, pp. 17312–17334, 2021.

[15] P. Yildirim, "Chronic kidney disease prediction on imbalanced data by multilayer perceptron," in *Proc. IEEE 41st Annual Computer Software and Applications Conference (COMPSAC)*, 2017, pp. 193–198.

[16] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017.

[17] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[18] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[19] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.

[21] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[22] Qwen Team, Alibaba Group, "Qwen3 Technical Report," 2025. Model: Qwen3-4B-Instruct-2507.

[23] E. J. Hu, Y. Shen, P. Wallis, et al., "LoRA: Low-rank adaptation of large language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[24] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[25] J. Chen, S. Xiao, P. Zhang, et al., "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," arXiv preprint arXiv:2402.03216, 2024.

[26] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[27] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[28] K. Singhal, S. Azizi, T. Tu, et al., "Large language models encode clinical knowledge," *Nature*, vol. 620, pp. 172–180, 2023.

[29] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, et al., "Large language models in medicine," *Nature Medicine*, vol. 29, pp. 1930–1940, 2023.

[30] I. Li, J. Pan, J. Goldwasser, et al., "Neural natural language processing for unstructured data in electronic health records: a review," *Computer Science Review*, vol. 46, p. 100511, 2022.

[31] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[32] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[33] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[34] J. P. Kincaid, R. P. Fishburne, R. L. Rogers, and B. S. Chissom, "Derivation of new readability formulas for Navy enlisted personnel," Naval Air Station Memphis, Research Branch Report 8-75, 1975.

[35] S. Ramírez, "FastAPI: A modern, fast web framework for building APIs with Python," 2018–2026. [Online]. Available: https://fastapi.tiangolo.com/

APPENDIX A

Input Feature Schema

Table A.1 gives the complete input specification enforced by the API and the web form: the accepted range, the reference value used by the perturbation explanation, and the normal range displayed to the user.

Table A.1: Input feature schema with validation ranges and explanation reference values.

Field	**Unit**	**Accepted range**	**Reference r_f**	**Type**
age	years	18–85	55.0	numeric
income_poverty_ratio	ratio	0.0–5.0	2.0	numeric
waist	cm	40–180	98.0	numeric
mean_sbp	mmHg	70–240	125.0	numeric
mean_dbp	mmHg	35–140	78.0	numeric
glucose	mg/dL	40–500	100.0	numeric
hba1c	%	3.0–18.0	5.6	numeric
blood_urea_nitrogen	mg/dL	2–150	14.0	numeric
uric_acid	mg/dL	1.0–15.0	5.5	numeric
serum_albumin	g/dL	1.5–6.0	4.2	numeric
hemoglobin	g/dL	5.0–20.0	14.0	numeric
diabetes_yes	0/1	{0, 1}	0	binary
hypertension_doctor_told	0/1	{0, 1}	0	binary
taking_bp_medicine	0/1	{0, 1}	0	binary

The API rejects unknown fields (`extra="forbid"`) and out-of-range values with HTTP 422 before the model is invoked. Fields left blank are imputed by the fitted preprocessing pipeline and counted in n_{miss}.

APPENDIX B

Safety Rules and Output Contract

B.1 The nine safety rules

The following rules appear verbatim in the full system prompt and are mirrored by the deployed validator.

1. Do not state or imply a diagnosis. Never say the person has CKD, kidney disease, or kidney failure. This is a screening-based risk level, not a diagnosis.
2. Do not prescribe, recommend, start, stop, change, or dose any medication or supplement.
3. Do not give individualised nutrition restrictions or numeric targets for protein, potassium, phosphorus, or sodium. If diet is relevant, suggest discussing a personalised plan with a clinician or registered/renal dietitian.
4. Do not replace clinician judgement, eGFR testing, or urine albumin (UACR) testing.
5. Give wellness education tailored to the patient's specific conditions and risk factors shown in the patient profile. Reference their actual conditions when giving food, activity, or monitoring advice; do not diagnose, prescribe, or treat.
6. For a moderate or high screening result, always include a reminder to confirm the result with a clinician and appropriate laboratory tests.
7. If the inputs suggest an urgent problem (for example chest pain, severe shortness of breath, severe swelling, or very high blood pressure), advise seeking prompt medical care.
8. Base the education on the provided guideline excerpts. If the excerpts do not cover something, keep the advice general, safe, and non-specific.
9. Use calm, supportive, non-alarming, plain language at about an eighth-grade reading level.

B.2 Output contract

Every response must contain exactly five labelled sections in the following order, followed by a machine-readable grounding line listing the identifiers of the guideline excerpts actually used:

1. **Summary** — names the screening risk level in everyday terms and the one or two factors that most influenced it.
2. **Food habits** — eating habits tied to this individual's factors; no numeric nutrient targets.
3. **Rest and sleep** — practical sleep and rest habits.
4. **Daily routine** — movement, hydration and monitoring habits chosen for this individual's factors.
5. **Follow-up reminder** — when and why to consult a clinician and which routine tests to ask about; mandatory for moderate and high.

APPENDIX C

Validator Detection Layers

Table C.1: Validator layers, severity and disposition.

Layer	Detects	Severity	Action
Structural check	A required section is absent	—	Regenerate once, then fall back
Diagnosis blocker	Assertion that the person has CKD, kidney disease or kidney failure	Severe	Safe template
Medication blocker	Drug names or classes, dosing expressions, or start/stop/change instructions	Severe	Safe template
Supplement blocker	Recommendation to take a supplement	Severe	Safe template
Nutrition restriction	Numeric individualised protein, potassium, phosphorus or sodium target	Moderate	Rewrite to dietitian referral
Lab-confirmation reminder	Moderate or high result lacking a clinician and laboratory follow-up mention	Moderate	Append reminder
Emergency redirect	Systolic $\geq$180 or diastolic $\geq$120 mmHg, or red-flag symptom vocabulary	Moderate	Prepend urgent-care advice

Detection is sentence-scoped and negation-aware throughout: a candidate match is discarded when governed by a negation or an explicit deferral to a clinician, so that safe guidance containing clinical vocabulary ("do not stop or change any medicine without talking to your doctor") is not misclassified as a violation.

APPENDIX D

Worked Example

The following illustrates a complete request for a high-risk profile. Values are representative of the benchmark's emergency subgroup.

D.1 Input

```
age 65, waist 112 cm, mean_sbp 186 mmHg, mean_dbp 104 mmHg,
glucose 168 mg/dL, hba1c 8.5%, blood_urea_nitrogen 26 mg/dL,
uric_acid 8.1 mg/dL, serum_albumin 3.6 g/dL, hemoglobin 11.9 g/dL,
income_poverty_ratio 1.2, diabetes_yes 1, hypertension_doctor_told
1, taking_bp_medicine 1
```

D.2 Screening output

```
stage1: p_atrisk = 0.7447 (≥ 0.44) → sent to stage 2
stage2: p_high_given_atrisk = 0.7290 (≥ 0.50)
risk_class = high
probabilities: low 0.2553, moderate 0.2018, high 0.5428
confidence = 0.5428, reliability = moderate
top factors: mean_sbp (raises), hba1c (raises),
    blood_urea_nitrogen (raises), waist (raises)
```

D.3 Retrieval

Priority topics derived from the risk-raising factors are `bp`, `diabetes`, `weight`, `followup`; four passages are selected, one per topic, drawn from KDIGO 2021 (blood pressure), the ADA/KDIGO consensus (diabetes), NKF patient education (weight and activity) and CDC prevention material (follow-up).

D.4 Validator action

Because systolic 186 mmHg exceeds the emergency threshold of 180 mmHg, the urgent-vitals alert block is inserted into the prompt, and the emergency-redirect layer verifies that urgent-care advice is present in the delivered text, prepending it otherwise. The disposition for this profile is `sanitized`, and the delivered Summary opens with an urgent-care instruction before restating the screening category.

APPENDIX E

Reproduction Instructions

E.1 Environment

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install -r requirements-llm.txt
```

E.2 Configuration

```
CKD_LLM_MODEL=qwen
CKD_LLM_ADAPTER=llm/finetune/adapters/v1/adapter
DATABASE_URL=postgresql+psycopg2://
    user:password@localhost:5432/ckd_research
CKD_DB_AUTO_CREATE=false
```

E.3 Pipeline commands

```
# database schema
python -m alembic upgrade head

# build the retrieval index
ollama pull bge-m3
python -m llm.rag.embed_index -embedder ollama-bge-m3

# generate fine-tuning data and train the adapter
python -m llm.data.generate_dataset -n 600 -out-dir phase7_data
python -m llm.finetune.qlora_train -data-dir phase7_data

# build the evaluation benchmark and run the ablation
python -m llm.eval.q1.benchmark \
    -out q1_benchmark/q1_profiles_300.jsonl
python -m llm.eval.q1.run_arms -arm A ...  -arm F
python -m llm.eval.q1.report -runs q1_runs -out-dir q1_report

# regenerate thesis figures
python thesis/make_figures.py

# run the application
uvicorn backend.main:app -reload
cd frontend && npm run dev
```

E.4 Verification

The health endpoint confirms which model is actually serving requests:

```
GET /generate-guidance/health
{
   "model_spec":
      "qwen:Qwen/Qwen3-4B-Instruct-2507+lora:v1/adapter",
   "adapter_active":  true,
   "prompt":  "compact",
   "mode":  "production"
}
```